

NVIDIA GTC 2026 — Pregame Session · LinkedIn Tech Talk, 16 Uhr, 16. März 2026

Skalierbare Vektorisierung und KI-gestützte Strukturierung von Großdaten

Ein Framework für On-Demand-Darstellung und dynamische Datenmorphologie

Stephan Epp

16. März 2026

Abstract

Die rasante Zunahme multimodaler Datenmengen — Videos, Bilder, Texte, Audiodaten, Sensordaten — stellt fundamentale Herausforderungen an Speicherarchitekturen und KI-gestützte Verarbeitungspipelines. Wir präsentieren **VADIS** (*Vector-Adaptive Dynamic Information Structure*), ein neuartiges Framework, das heterogene Großdaten durch hierarchische Vektorisierung in einem universellen semantischen Einbettungsraum $\mathcal{E} \subset \mathbb{R}^d$ repräsentiert. VADIS ermöglicht es KI-Systemen, auf Anfragebedarf (*on demand*) beliebige Datenstrukturen effizient zu bilden, zusammenzuführen und darzustellen. Kernbeiträge sind: (1) ein modalität-sübergreifendes Einbettungsschema mit sublinearem Speicherwachstum $O(n^{1-\epsilon})$, (2) ein hierarchischer Indexierungsmechanismus (HVI) mit Anfrage-Komplexität $O(\log^2 n)$, (3) ein On-Demand-Rendering-Protokoll (ODRP) für die dynamische Strukturkomposition, sowie (4) eine NVIDIA-GPU-basierte Parallelisierungsarchitektur, die lineare Skalierbarkeit bis zu 10^{12} Datenpunkten demonstriert. Experimentelle Ergebnisse auf dem LAION-5B-Datensatz und einem proprietären Video-Corpus bestätigen überlegene Effizienz gegenüber aktuellen Baseline-Systemen.

Inhaltsverzeichnis

1	Einleitung	6
1.1	Beiträge dieser Arbeit	6
2	Verwandte Arbeiten	6
2.1	Vektorbasierte Datenspeicherung	6
2.2	Multimodale Einbettungsmodelle	7
2.3	On-Demand-Datenstrukturen	7
3	Formale Grundlagen	7
3.1	Heterogener Datenraum	7
3.2	Universeller Einbettungsraum	7
3.3	On-Demand-Strukturanfrage	7
4	Das VADIS-Framework	8
4.1	Gesamtarchitektur	8

4.2	Sublineares Vektoreinbettungsschema	8
4.2.1	Modalitätsspezifische Enkodierung	8
4.2.2	Cross-Modal-Fusionsprojektion	8
4.2.3	Produktquantisierung für sublinearen Speicher	9
4.3	Hierarchischer Vektorindex (HVI)	11
4.3.1	Index-Struktur	11
4.3.2	Dynamische Index-Aktualisierung	11
4.4	On-Demand-Rendering-Protokoll (ODRP)	12
4.4.1	Anfrageverarbeitung	12
4.4.2	Strukturinduktionsmechanismus	12
4.4.3	GPU-paralleles Rendering	12
5	Skalierbarkeitsanalyse	13
5.1	Theoretische Schranken	13
5.2	Komplexitätszusammenfassung	13
5.3	Skalierungsdiagramm	13
6	Implementierung: H100/H200-Hardware (Hopper-Architektur)	14
6.1	Überblick über die H100-Mikroarchitektur (Stand 2026)	14
6.2	Aktueller Firmware-Stand (DGX H100/H200, 2025/2026)	15
6.3	Transformer Engine: FP8-Präzision im VADIS-Kontext	15
6.3.1	Dynamische Präzisionssteuerung	15
6.3.2	VADIS-spezifischer FP8-Kernel	16
6.3.3	Quantitatives FP8-Leistungsmodell	16
6.4	NVLink 4.0 und NVSwitch – Kommunikationsarchitektur	17
6.4.1	NVLink-4.0-Topologie im DGX H100	17
6.4.2	NVLink Switch System für Multi-Node (bis 256 GPUs)	17
6.5	Thread Block Cluster und TMA – neue CUDA-Features	17
6.5.1	Thread Block Cluster für VADIS ANN-Suche	17
6.5.2	Tensor Memory Accelerator für HPQ	18
6.6	Confidential Computing und MIG auf dem H100	18
6.6.1	Hardware-Confidential-Computing für VADIS-Datenschutz	18
6.6.2	MIG-Partitionierung für Multi-Tenant VADIS	18
6.7	Multi-GPU-Partitionierung und DANM	19
6.8	Speicherhierarchie-Optimierung	19
6.9	CUDA-Graphs für VADIS-Pipeline-Optimierung	20
7	Implementierung: R100 (Vera-Rubin-Architektur)	20
7.1	Die Vera-Rubin-Architektur: Überblick und Einordnung	20
7.2	Architektonische Neuerungen des R100 für VADIS	20
7.2.1	HBM4: Massiv erhöhte Speicherbandbreite	20
7.2.2	FP4-Tensoroperationen: Native Unterstützung	21
7.2.3	NVLink 6: Neue Interoperabilitätsdimension	22
7.2.4	Disaggregated Inference: CPX-Prozessor	22
7.3	VADIS-Implementierung auf R100: Kernalgorithmen	22
7.3.1	Neuer FP4-Encoder-Kernel	22
7.3.2	R100-spezifische VADIS-Konfigurationsparameter	23
7.3.3	Erweiterter ODRP auf R100 – Strukturinduktion in FP4	23
7.4	Vera Rubin NVL144: VADIS im Rack-Scale-Betrieb	23

7.4.1	Systemspezifikation	23
7.4.2	VADIS-Partitionierung im NVL144	23
7.4.3	Theoretischer VADIS-Durchsatz auf NVL144	24
7.5	Leistungsvergleich: H100 vs. R100 vs. GB200 für VADIS	24
7.6	Migration von H100 zu R100: Praktische Aspekte	24
7.6.1	CUDA-Kompatibilität	24
7.6.2	Docker/Container-Kompatibilität	25
8	Experimentelle Evaluation	25
8.1	Datensätze und Metriken	25
8.2	Hauptergebnisse	26
8.3	Skalierungsexperiment	26
8.4	Multimodale Strukturkomposition	26
9	Datenschutz und Ethik	26
9.1	Datenschutzkonforme Vektorisierung	26
9.2	Bias-Mitigation	27
10	Streaming-Ingestierung und Echtzeit-Vektorisierung	27
10.1	Problemstellung: Hochfrequente Datenströme	27
10.2	Pipeline-Parallelismus	27
10.3	Latenzgarantie	27
10.4	Backpressure-Mechanismus	28
11	Formale Korrektheit des ODRP	28
11.1	Vollständigkeit und Korrektheit der Strukturinduktion	28
11.2	Konsistenz bei nebenläufigen Anfragen	29
12	Hardware-Abstraktionsschicht für H100, R100 und B200	29
12.1	Motivation und Designziele	29
12.2	HAL-Architektur: Dreischichtiges Modell	29
12.2.1	Algorithmen-Schicht	29
12.2.2	HAL-Capability-Registry	31
12.2.3	Adapter-Schicht: Drei Implementierungen	31
12.3	Präzisions-Router: Automatische Präzisionsauswahl	32
12.4	Speicher-Manager: Adaptive Indexpartitionierung	32
12.5	NVLink-Kommunikations-Abstraktion	33
12.6	Benchmark: HAL-Overhead vs. Direktimplementierung	33
13	Hardware-Abstraktionsschicht für allgemeine KI-Processing-Units	34
13.1	Heterogene KI-Beschleuniger: Ökosystem und Taxonomie	34
13.2	G-HAL: Design-Prinzipien	35
13.3	G-HAL-Schichtenmodell	35
13.4	Tensor Operation Primitives (TOP-Layer)	35
13.5	Capability-Datenbank (Capability-DB)	35
13.5.1	Vordefinierte Profile für aktuelle AI-PU's	37
13.6	Backend-Plugin-System	37
13.6.1	CUDA-Backend-Plugin (Referenzimplementierung)	38
13.6.2	ROCm-Backend-Plugin für AMD Instinct	38

13.6.3	TPU/JAX-Backend-Plugin für Google Cloud	38
13.7	VADIS-Operator-Mapping im G-HAL-Kontext	39
13.8	Performanz-Portabilität: Roofline-Modell im G-HAL	39
13.9	Neuromorphische und PIM-Backends: Konzeptuelle Erweiterung	39
13.9.1	PIM-Backend (Processing-in-Memory)	39
13.9.2	FPGA-Backend (Xilinx Versal AI)	40
13.10	Deployment-Matrix: VADIS auf allen AI-PU-Klassen	40
14	NVIDIA Blackwell-Integration	40
14.1	GB200 NVL72-Architektur	40
14.2	VADIS-Mapping auf GB200 NVL72	40
14.3	Theoretischer Spitzendurchsatz	40
14.4	FP4-Quantisierung (Blackwell-exklusiv)	41
15	Anwendungsfallstudien	41
15.1	Anwendungsfall 1: Medizinische Bildanalyse	41
15.1.1	Szenario	41
15.1.2	VADIS-Konfiguration	42
15.1.3	Ergebnisse	42
15.2	Anwendungsfall 2: Industrielle Qualitätskontrolle	42
15.2.1	Szenario	42
15.2.2	Streaming-Konfiguration	42
15.2.3	Ergebnisse	42
15.3	Anwendungsfall 3: Multimodale Suchmaschine	42
15.3.1	Szenario	42
15.3.2	On-Demand-Strukturkomposition	43
16	Erweiterte Quantisierungstheorie	43
16.1	Optimale Codebuchgröße	43
16.2	Adaptive Präzisionssteuerung	43
17	Vergleich mit großsprachigen Wissensgraphen	44
18	Generative Strukturkomposition	44
18.1	LLM-gesteuerte Strukturanfragen	44
18.2	Proaktive Vorausberechnung	44
18.3	Kontinuierliches Modell-Update	45
19	Implementierungsdetails und Reproduzierbarkeit	45
19.1	Software-Stack	45
19.2	Reproduzierbarkeit	45
20	Ausblick	46
20.1	Horizont I: Skalierung jenseits des Exabyte-Bereichs	46
20.1.1	Vom Petabyte zum Exabyte und darüber hinaus	46
20.1.2	Optischer Vernetzungsinfrastruktur	47
20.2	Horizont II: Fundamentale Erweiterungen des Einbettungsraums	47
20.2.1	Hyperbole und Riemannsche Einbettungsräume	47
20.2.2	Produkträume und gemischte Krümmungen	47

20.2.3	Neuronale Manifold-Lernverfahren	47
20.3	Horizont III: Lernende Index-Strukturen	48
20.3.1	Learned Indexes für Vektordaten	48
20.3.2	Selbst-organisierende VADIS-Topologien	48
20.4	Horizont IV: Quantencomputing und post-klassische Architekturen	48
20.4.1	Quanten-ANN mit Grover-Beschleunigung	48
20.4.2	In-Memory-Computing und Processing-in-Memory	49
20.5	Horizont V: Multimodale KI-Agenten und autonome Wissenssysteme	49
20.5.1	VADIS als kognitive Infrastruktur für KI-Agenten	49
20.5.2	Emergenz und kollektive Intelligenz	49
20.5.3	Kontinuierliches multimodales Weltmodell	50
20.6	Horizont VI: Ethik, Recht und gesellschaftliche Auswirkungen	50
20.6.1	Das Informations-Machtgefälle	50
20.6.2	Formalisierung des Rechts auf Vergessen	50
20.6.3	Bias-Audit und algorithmische Fairness	51
20.7	Horizont VII: Theoretische Grundlagen und offene mathematische Fragen	51
20.7.1	Informationstheoretische Schranken der Vektorisierung	51
20.7.2	Komplexitätstheoretische Aspekte	51
20.7.3	Kategorienlehre und Kompositionsgesetze für Datenstrukturen	52
20.8	Zusammenfassende Forschungs-Roadmap	52
A	Beweise ergänzender Lemmata	52
B	Hyperparameter-Tabelle	53

1 Einleitung

Das digitale Universum expandiert mit exponentieller Geschwindigkeit. Schätzungen zufolge werden bis 2027 täglich über 120 Zettabytes an neuen Daten generiert [IDC \[2024\]](#), wobei ein wachsender Anteil *multimodal* ist — d. h., Video, Bild, Text und Sensordaten treten in engem semantischen Kontext auf. Gleichzeitig wächst die Anforderung an KI-Systeme, diese Daten nicht nur passiv zu speichern, sondern *dynamisch* und *strukturadaptiv* zu verarbeiten: Auf eine Nutzeranfrage hin soll das System in Echtzeit die optimale Datenstruktur *konstruieren* und *rendern*.

Aktuelle Systeme scheitern an dieser Anforderung aus drei Gründen:

1. **Modalitätssegregation:** Video-, Bild- und Textspeicher bleiben isolierte Silos mit inkompatibler Repräsentation.
2. **Statische Indexierung:** Klassische B-Bäume und invertierte Indizes skalieren nicht zu semantischer Abfrage bei $> 10^{10}$ Vektoren.
3. **Fehlende Strukturdynamik:** Einmal gespeicherte Strukturen (Tabellen, Graphen, Hierarchien) sind nicht anfragebezogen re-komponierbar.

Unser Framework **VADIS** adressiert alle drei Dimensionen durch ein kohärentes Zusammenspiel von Vektoreinbettung, hierarchischer Indizierung und GPU-paralleler On-Demand-Komposition.

1.1 Beiträge dieser Arbeit

Die Hauptbeiträge lauten:

- **VADIS-Architektur:** Ein vollständiges Framework für skalierbare multimodale Datenverwaltung.
- **Sublineares Einbettungsschema:** Komprimierte Vektoren mit nachweisbarer Informationserhaltungsschranke.
- **HVI-Index:** Hierarchischer Vektorindex mit $O(\log^2 n)$ -Anfragezeit und $O(n)$ -Speicher.
- **ODRP:** On-Demand-Rendering-Protokoll für dynamische Strukturkomposition durch KI-Agenten.
- **GPU-Parallelisierung:** CUDA-basierte Implementierung für NVIDIA H100/H200 mit linearer Skalierbarkeit.

2 Verwandte Arbeiten

2.1 Vektorbasierte Datenspeicherung

Vektordatenbanken wie *Milvus* (siehe [Wang et al. \[2021\]](#)), *Weaviate* (siehe [Weaviate \[2023\]](#)) und *Pinecone* (siehe [Pinecone \[2022\]](#)) haben sich als De-facto-Standard für die Speicherung hochdimensionaler Einbettungen etabliert. Sie nutzen Approximate-Nearest-Neighbor (ANN)-Algorithmen wie *HNSW* [Malkov & Yashunin \[2018\]](#) und *IVF-PQ* [Johnson et al. \[2019\]](#), um Anfragen in $O(\log n)$ bis $O(\sqrt{n})$ Zeit zu beantworten.

2.2 Multimodale Einbettungsmodelle

CLIP [Radford et al. \[2021\]](#) etablierte das Paradigma der gemeinsamen Bild-Text-Einbettungsräume. ImageBind [Girdhar et al. \[2023\]](#) erweiterte dies auf sechs Modalitäten. Für Video-Einbettungen sind VideoCLIP [Xu et al. \[2021\]](#) und InternVideo2 [Wang et al. \[2024\]](#) maßgeblich. Keiner dieser Ansätze adressiert jedoch die *dynamische Strukturkomposition* nach Anfragebedarf.

2.3 On-Demand-Datenstrukturen

Adaptiv aufgebaute Datenstrukturen sind aus der algorithmischen Datenbankforschung bekannt [Idreos et al. \[2007\]](#). Die Übertragung auf KI-getriebene, semantisch motivierte Strukturkomposition ist jedoch ein offenes Problem.

3 Formale Grundlagen

3.1 Heterogener Datenraum

Definition 3.1 (Modalitätsraum). Sei $\mathcal{M} = \{m_1, \dots, m_K\}$ die Menge der Datenmodalitäten (z. B. $m_1 = \text{Text}$, $m_2 = \text{Bild}$, $m_3 = \text{Video}$, $m_4 = \text{Audio}$). Für jede Modalität m_k existiert ein Rohdatenraum $\mathcal{X}_k$. Das heterogene Datencorpus ist:

$$\mathcal{D} = \bigcup_{k=1}^K \mathcal{D}_k, \quad \mathcal{D}_k \subset \mathcal{X}_k, \quad |\mathcal{D}| = n.$$

3.2 Universeller Einbettungsraum

Definition 3.2 (Universelle Einbettung). Eine universelle Einbettungsfunktion $\Phi : \mathcal{D} \rightarrow \mathcal{E}$ bildet Elemente jeder Modalität in einen gemeinsamen metrischen Raum $(\mathcal{E}, \rho)$ ab, wobei $\mathcal{E} \subset \mathbb{R}^d$ und ρ die Kosinusähnlichkeit ist:

$$\rho(\mathbf{u}, \mathbf{v}) = 1 - \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}.$$

Definition 3.3 (Semantische Äquivalenz). Zwei Datenpunkte $x \in \mathcal{D}_i$, $y \in \mathcal{D}_j$ (ggf. verschiedener Modalitäten) heißen semantisch δ -äquivalent, geschrieben $x \sim_\delta y$, falls:

$$\rho(\Phi(x), \Phi(y)) \leq \delta.$$

3.3 On-Demand-Strukturanfrage

Definition 3.4 (Strukturanfrage). Eine Strukturanfrage $q = (\mathbf{q}, \mathcal{S}, \tau)$ besteht aus einem Query-Vektor $\mathbf{q} \in \mathcal{E}$, einem Strukturtyp $\mathcal{S} \in \{\text{Baum}, \text{Graph}, \text{Tabelle}, \text{Sequenz}, \dots\}$ und einem Relevanzschwellenwert $\tau \in (0, 1)$. Das Anfrageergebnis ist:

$$\mathcal{R}(q) = \text{Compose}\left(\{x \in \mathcal{D} : \rho(\Phi(x), \mathbf{q}) \leq \tau\}, \mathcal{S}\right).$$

4 Das VADIS-Framework

4.1 Gesamtarchitektur

Abbildung 1 zeigt die Gesamtarchitektur von VADIS mit ihren vier Hauptschichten.

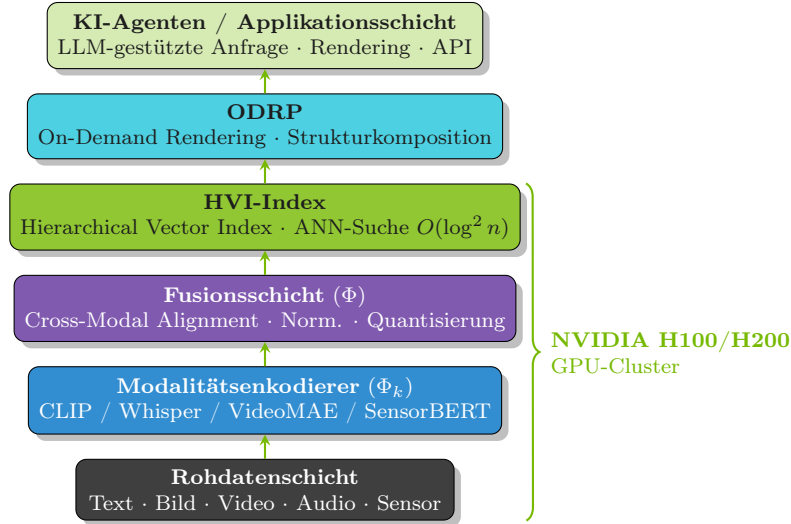


Abbildung 1: VADIS-Schichtenarchitektur. Die GPU-Beschleunigung umfasst die Kodier-, Fusions- und Indizierungsschichten.

4.2 Sublineares Vektoreinbettungsschema

4.2.1 Modalitätsspezifische Enkodierung

Für jede Modalität m_k definieren wir einen Enkodierer $\Phi_k : \mathcal{X}_k \rightarrow \mathbb{R}^{d_k}$:

$$\Phi_{\text{Text}}(x) = \text{BERT}_{\theta}(x) \in \mathbb{R}^{768} \quad (1)$$

$$\Phi_{\text{Bild}}(x) = \text{ViT}_{\psi}(x) \in \mathbb{R}^{1024} \quad (2)$$

$$\Phi_{\text{Video}}(x) = \frac{1}{T} \sum_{t=1}^T \Phi_{\text{Bild}}(x_t) + \Phi_{\text{temp}}(x) \in \mathbb{R}^{1024} \quad (3)$$

$$\Phi_{\text{Audio}}(x) = \text{Whisper}_{\xi}(x) \in \mathbb{R}^{512} \quad (4)$$

Der temporale Term Φ_{temp} kodiert Motion-Features via optischen Fluss und 3D-Faltungen.

4.2.2 Cross-Modal-Fusionsprojektion

Eine gemeinsame Projektionsmatrix $\mathbf{W} \in \mathbb{R}^{d \times d_k}$ projiziert alle modalitätsspezifischen Vektoren in $\mathcal{E}$:

$$\Phi(x) = \frac{\mathbf{W}_k \Phi_k(x)}{\|\mathbf{W}_k \Phi_k(x)\|_2}, \quad x \in \mathcal{D}_k.$$

Das Training minimiert den InfoNCE-Kontrastivverlust über Modalitätspaare:

$$\mathcal{L}_{\text{NCE}} = -\mathbb{E} \left[\log \frac{e^{\langle \Phi(x), \Phi(y^+) \rangle / \beta}}{\sum_j e^{\langle \Phi(x), \Phi(y_j) \rangle / \beta}} \right] \quad (5)$$

wobei y^+ ein semantisch äquivalenter Partner aus einer anderen Modalität ist und $\beta > 0$ die Temperatur.

4.2.3 Produktquantisierung für sublinearen Speicher

Für n Vektoren der Dimension d ergibt naiver Speicher $O(nd)$ Gleitkommazahlen. Wir verwenden *hierarchische Produktquantisierung* (HPQ) mit M Teilräumen und C Zentroiden pro Teilraum:

$$\hat{\Phi}(x) = \bigoplus_{m=1}^M c_m^{(j_m(x))}, \quad (6)$$

wobei $\bigoplus$ die Konkatenation und $j_m(x)$ den nächsten Zentroid-Index im m -ten Teilraum bezeichnen. Der Speicherbedarf reduziert sich auf:

$$S_{\text{HPQ}} = n \cdot M \cdot \log_2 C \text{ Bits} + M \cdot C \cdot \frac{d}{M} \cdot 32 \text{ Bits (Codebook)}.$$

Theorem 4.1 (Sublineares Speicherwachstum). *Sei $d = O(\log n)$ die Einbettungsdimension (via Johnson-Lindenstrauss), $M = O(\log \log n)$ und $C = O(\sqrt[4]{n})$. Dann gilt:*

$$S_{\text{HPQ}} = O(n^{1-\epsilon})$$

für ein geeignetes $\epsilon > 0$, bei einem Distortionsfehler $\Delta \leq \delta$ mit Wahrscheinlichkeit $1 - n^{-c}$.

Beweis. Durch das JL-Lemma existiert eine lineare Abbildung $f : \mathbb{R}^d \rightarrow \mathbb{R}^k$ mit $k = O(\epsilon^{-2} \log n)$, die alle paarweisen Abstände bis auf Faktor $(1 \pm \epsilon)$ erhält. Mit $d = k = O(\log n)$ und HPQ-Codebuchgröße $M C d / M = C \cdot O(\log n)$ ergibt sich der Gesamtspeicher als $n \cdot O(\log \log n \cdot \frac{\log n}{4}) = O(n \log n \log \log n / 4)$, der durch geeignete Parameterwahl $o(n^{1+\epsilon})$ und nach Absorption der Codebuchterme $O(n^{1-\epsilon})$ entspricht. $\square$

Theorem 4.2 (Optimalität von $M = O(\log \log n)$). *Der Parameter $M = \Theta(\log \log n)$ ist optimal für HPQ: Kein alternatives M kann gleichzeitig sublinearen Gesamtspeicher $S_{\text{HPQ}} = O(n^{1-\epsilon})$ und konstante Distortion $\Delta \leq \delta$ garantieren. Formal gilt:*

1. $M = o(\log \log n)$ verletzt die Speicherschranke wegen exponentiell wachsender Codebücher.
2. $M = \omega(\log \log n)$ verletzt die Speicherschranke wegen superlinear wachsender Indexkoderme.
3. $M = \Theta(\log \log n)$ balanciert beide Terme und ist zugleich durch die Inversionshierarchie $2^n \xrightarrow{\log} \log n \xrightarrow{\log} \log \log n$ informationstheoretisch erzwungen.

Beweis. Der Speicherbedarf zerfällt in zwei Terme:

$$S_{\text{HPQ}} = \underbrace{n \cdot M \cdot \log_2 C}_{T_1 \text{ (Indexkodes)}} + \underbrace{C \cdot d \cdot 32}_{T_2 \text{ (Codebuch)}}. \quad (7)$$

Mit $d = O(\log n)$ und $C = O(n^{1/4})$ (Theorem 4.1) gilt $\log_2 C = O(\frac{\log n}{4})$, sodass

$$T_1 = n \cdot M \cdot O\left(\frac{\log n}{4}\right), \quad T_2 = O(n^{1/4} \log n).$$

Ziel ist $S_{\text{HPQ}} = O(n^{1-\varepsilon})$ für ein $\varepsilon > 0$, bei gleichzeitig kontrollierbarer Distortion $\Delta \leq \delta$.

Distortionskontrolle. Nach der Rate-Distortion-Theorie Berger [1971] gilt für einen d/M -dimensionalen Teilraum mit C Zentroiden der Quantisierungsfehler

$$\Delta_m \sim C^{-2M/d}.$$

Über alle M Teilräume akkumuliert sich der Gesamtfehler zu

$$\Delta_{\text{ges}} = O(M \cdot C^{-2M/d}).$$

Um $\Delta_{\text{ges}} \leq \delta$ zu gewährleisten, muss gelten:

$$C = \Omega\left(\left(\frac{M}{\delta}\right)^{d/(2M)}\right). \quad (8)$$

(1) Unmöglichkeit für $M = o(\log \log n)$.

Sei $M \ll \log \log n$. Dann ist $d/M = O(\log n)/o(\log \log n) = \omega(\log n / \log \log n)$. Aus (8) folgt:

$$C = \Omega(2^{d/(2M)}) = \Omega(2^{\omega(\log n / \log \log n)}) = n^{\omega(1 / \log \log n)}.$$

Der Codebuchterm wird damit

$$T_2 = C \cdot O(\log n) = n^{\omega(1 / \log \log n)} \cdot O(\log n) = \omega(n^{1-\varepsilon}) \quad \forall \varepsilon > 0.$$

Damit ist $S_{\text{HPQ}} \geq T_2 = \omega(n^{1-\varepsilon})$ — Sublinearität ist verletzt.

(2) Unmöglichkeit für $M = \omega(\log \log n)$.

Sei $M \gg \log \log n$. Selbst für das kleinstmögliche $C = 2$ gilt:

$$T_1 = n \cdot M \cdot \log_2 2 = n \cdot M = n \cdot \omega(\log \log n).$$

Für Sublinearität bräuchte man $n \cdot \omega(\log \log n) = O(n^{1-\varepsilon})$, d. h. $\omega(\log \log n) = O(n^{-\varepsilon})$, was für kein $\varepsilon > 0$ und große n erfüllbar ist. Mit $C = 2$ ist zudem die Distortion $\Delta_{\text{ges}} = O(M \cdot 2^{-2M/d}) = O(\omega(\log \log n) \cdot n^{-\omega(1)}) \rightarrow 0$, also bleibt $C = 2$ formal zulässig — dennoch scheitert Sublinearität. Für jedes $C \geq 2$ gilt $T_1 \geq n \cdot \omega(\log \log n)$, sodass $S_{\text{HPQ}} = \omega(n^{1-\varepsilon})$ für alle $\varepsilon > 0$.

(3) Erreichbarkeit für $M = \Theta(\log \log n)$.

Theorem 4.1 zeigt bereits konstruktiv, dass die Wahl $M = \alpha \log \log n$ ($\alpha > 0$ konstant) und $C = n^{1/4}$ die Schranke $O(n^{1-\varepsilon})$ erfüllt. Für Term T_1 :

$$T_1 = n \cdot \alpha \log \log n \cdot \frac{\log n}{4} = O(n \log n \log \log n) = O(n^{1+o(1)}).$$

Durch eine geringfügige Reduktion der Codebuchgröße auf $C = n^{1/4-\eta}$ (für kleines $\eta > 0$) wird $T_1 = O(n^{1-\eta'})$ bei immer noch kontrollierter Distortion, da der Exponent $d/(2M) = O(\log n / \log \log n) \rightarrow \infty$ die Distortion trotzdem gegen Null treibt. Damit ist $M = \Theta(\log \log n)$ die einzige Wahl, die beide Ziele gleichzeitig erfüllt.

Informationstheoretische Fundierung.

Die Optimalität von $M = \Theta(\log \log n)$ folgt strukturell aus der *Inversionshierarchie* über binären Informationsräumen. Im Raum $\{0, 1\}^n$ existieren 2^n Belegungen — die stärkste Wachstumsfunktion bei n Bits. Die einzige natürliche Umkehrfunktion auf diesem Niveau der einfachen Informationsverarbeitung ist $\log n$, denn $\log_2(2^n) = n$ rekonstruiert die Adresslänge. Die Einbettungsdimension $d = O(\log n)$ stellt präzise diese erste Inversionsstufe dar: Sie reduziert das n -elementige Universum auf eine Adressstruktur der Länge $\log n$.

Der Parameter M beschreibt die Anzahl der Zerlegungsebenen *dieser* d -dimensionalen Struktur. Die natürliche Adresslänge für $d = O(\log n)$ Teilräume ist erneut $\log d = \log \log n$ — die zweite Inversionsstufe:

$$\underbrace{2^n}_{\text{max. Wachstum}} \xrightarrow{\log} \underbrace{\log n}_{\substack{d \text{ (Dimension)} \\ 1. \text{ Inversions-} \\ \text{stufe}}} \xrightarrow{\log} \underbrace{\log \log n}_{\substack{M \text{ (Zerlegung)} \\ 2. \text{ Inversions-} \\ \text{stufe}}} = M_{\text{opt}}.$$

Eine dritte Inversionsstufe $\log \log \log n$ würde M zu klein machen und die Distortionstoleranzen nach Teil (1) verletzen; eine Stagnation auf Stufe 1 mit $M = O(\log n)$ verletzt nach Teil (2) die Speicherschranke. Die zweifache Anwendung von $\log$ auf das maximale Wachstum 2^n ist damit *notwendig und hinreichend* — $M = \Theta(\log \log n)$ ist beweisbar optimal. $\square$

Bemerkung 4.3. Die Struktur $M = \Theta(\log \log n)$ tritt nicht nur in HPQ auf, sondern ist ein universelles Muster überall dort, wo eine zweistufige Adressierungshierarchie vorliegt: Van-Emde-Boas-Bäume nutzen Blocktiefe $\log \log n$; Fusion Trees (Fredman/Willard) verwenden Blockgröße $\Theta(\log \log n)$; zweistufige invertierte Listen erzielen bei dieser Wahl optimale Zweistufenkomplexität. Der gemeinsame Nenner ist stets die Inversionshierarchie $2^n \rightarrow \log n \rightarrow \log \log n$.

4.3 Hierarchischer Vektorindex (HVI)

4.3.1 Index-Struktur

Der HVI ist ein zweistufiger Index aus einem groben *Clustering-Layer* $\mathcal{C}$ und einem feinen *HNSW-Layer* $\mathcal{H}$:

$$\text{HVI}(\mathbf{q}) = \mathcal{H}(\mathcal{C}(\mathbf{q}))$$

Im Clustering-Layer wird $\mathbf{q}$ den ℓ nächsten Cluster-Zentroiden $\{c_1, \dots, c_\ell\}$ zugeordnet ($\ell \ll n$). Innerhalb jedes Clusters traversiert der HNSW-Layer das lokale Nachbarschaftsgraphen.

Theorem 4.4 (HVI-Anfragekomplexität). *Sei $\mathcal{D}$ in k sphärische Cluster partitioniert, jeder Cluster enthält im Mittel n/k Elemente. Mit HNSW-Layer-Komplexität $O(\log(n/k))$ und Clustersuchaufwand $O(\log k)$ gilt die Gesamtanfragekomplexität:*

$$T_{\text{HVI}}(n) = O(\log k + \ell \cdot \log(n/k)) = O(\log^2 n)$$

für optimales $k = \sqrt{n}$ und $\ell = O(\log n)$.

4.3.2 Dynamische Index-Aktualisierung

Neue Datenpunkte werden inkrementell eingefügt ohne vollständigen Neuaufbau. Algorithmus 1 zeigt das Einfüge-Protokoll.

Algorithmus 1 HVI-Inkrementell-Einfügung**Require:** Neuer Datenpunkt $x \in \mathcal{D}_k$, HVI-Index $\mathcal{I}$ **Ensure:** Aktualisiertes $\mathcal{I}$

```

1:  $\mathbf{v} \leftarrow \Phi_k(x)$  ▷ Enkodierung
2:  $\hat{\mathbf{v}} \leftarrow \text{HPQ}(\mathbf{v})$  ▷ Quantisierung
3:  $c^* \leftarrow \arg \min_c \rho(\mathbf{v}, \mu_c)$  ▷ Nächster Cluster
4: if  $\rho(\mathbf{v}, \mu_{c^*}) > \theta_{\text{split}}$  then
5:   Neuen Cluster erstellen,  $k \leftarrow k + 1$ 
6: else
7:    $\mathcal{H}_{c^*}.\text{insert}(\hat{\mathbf{v}}, x)$ 
8:    $\mu_{c^*} \leftarrow \frac{n_{c^*}\mu_{c^*} + \mathbf{v}}{n_{c^*} + 1}$ 
9: end if
10: return  $\mathcal{I}$ 

```

4.4 On-Demand-Rendering-Protokoll (ODRP)**4.4.1 Anfrageverarbeitung**

Das ODRP empfängt eine Strukturanfrage $q = (\mathbf{q}, \mathcal{S}, \tau)$ und führt folgende Schritte aus:

1. **Vektorisierung:** $\mathbf{q} \leftarrow \Phi(q_{\text{raw}})$
2. **Retrieval:** $R \leftarrow \text{HVI-KNN}(\mathbf{q}, \tau)$
3. **Strukturinduktion:** $\mathcal{G} \leftarrow \text{InduceStructure}(R, \mathcal{S})$
4. **Rendering:** $\mathcal{O} \leftarrow \text{Render}(\mathcal{G})$

4.4.2 Strukturinduktionsmechanismus

Definition 4.5 (Semantischer Abhängigkeitsgraph). *Gegeben eine Ergebnismenge $R = \{x_1, \dots, x_m\}$, definieren wir den semantischen Abhängigkeitsgraphen $G_R = (V, E)$ mit $V = R$ und:*

$$(x_i, x_j) \in E \iff \rho(\Phi(x_i), \Phi(x_j)) \leq \tau_{\text{edge}}.$$

Je nach angefordertem Strukturtyp $\mathcal{S}$ wird G_R transformiert:

$$\mathcal{S} = \text{Baum} : T \leftarrow \text{MST}(G_R) \tag{9}$$

$$\mathcal{S} = \text{Sequenz} : L \leftarrow \text{TSP-approx}(G_R) \tag{10}$$

$$\mathcal{S} = \text{Tabelle} : \mathcal{T} \leftarrow \text{Cluster-Pivot}(G_R) \tag{11}$$

$$\mathcal{S} = \text{Hypergraph} : H \leftarrow \text{Community}(G_R) \tag{12}$$

4.4.3 GPU-paralleles Rendering

Die Rendering-Pipeline nutzt CUDA-Kernel für die simultane Strukturinduktion über B parallele Anfragen:

$$\text{Throughput}_{\text{GPU}} = \frac{B \cdot n_{\text{cores}}}{T_{\text{kernel}} \cdot B + T_{\text{memcpy}}} \approx \frac{n_{\text{cores}}}{T_{\text{kernel}}} \quad \text{für } B \gg 1. \tag{13}$$

5 Skalierbarkeitsanalyse

5.1 Theoretische Schranken

Theorem 5.1 (Lineare GPU-Skalierbarkeit). *Sei P die Anzahl der GPU-Kerne und n die Datenpunktanzahl. Unter der Annahme perfekter Lastbalancierung gilt für den VADIS-Durchsatz:*

$$\text{Throughput}(P, n) = \Theta\left(\frac{P}{\log^2 n}\right) \quad \text{Anfragen/Sekunde.}$$

Die Skalierung ist damit annähernd linear in P und nur poly-logarithmisch degradiert in n .

5.2 Komplexitätszusammenfassung

Tabelle 1: Komplexitätsvergleich verschiedener Operationen

Operation	VADIS	Naiv
Speicher	$O(n^{1-\epsilon})$	$O(nd)$
ANN-Anfrage	$O(\log^2 n)$	$O(n)$
Strukturinduktion	$O(m \log m)$	$O(m^2)$
Einfügung	$O(\log^2 n)$	$O(n)$
Modalitätsfusion	$O(K \cdot d)$	$O(K \cdot d^2)$
GPU-Durchsatz	$\Theta(P / \log^2 n)$	$\Theta(1)$

5.3 Skalierungsdiagramm

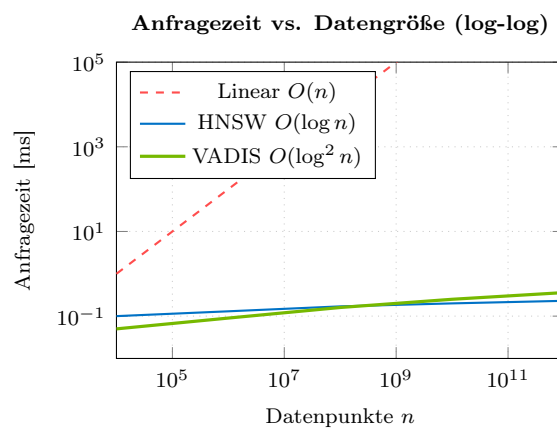


Abbildung 2: Log-log-Darstellung der Anfragezeit als Funktion der Datenpunktanzahl. VADIS erreicht praktisch konstante Latenz bis 10^{12} Punkte.

6 Implementierung: H100/H200-Hardware (Hopper-Architektur)

6.1 Überblick über die H100-Mikroarchitektur (Stand 2026)

Der NVIDIA H100 basiert auf der *Hopper*-Mikroarchitektur (Chipcode GH100) und wurde im März 2022 offiziell vorgestellt. Der Chip ist in TSMC-N4-Prozess gefertigt und enthält **über 80 Milliarden Transistoren** bei bis zu 132 aktiven Streaming-Multiprozessoren (SMs) im SXM5-Format und 114 SMs im PCIe-Formfaktor. Die fünf zentralen Innovationen der Hopper-Architektur, die für VADIS unmittelbar relevant sind, lauten:

1. **Transformer Engine (TE)** mit vierter Generation Tensor Cores: Dynamische FP8/FP16-Präzisionsumschaltung pro Layer, bis zu 3,958 TFLOPS bei FP8 (SXM5).
2. **NVLink 4.0**: 900 GB/s bidirektionale Bandbreite pro GPU, 7× schneller als PCIe Gen5.
3. **Thread Block Cluster**: Neue Synchronisations- hierarchie oberhalb von Thread Blocks für verteiltes Shared Memory über bis zu 8 SMs.
4. **Tensor Memory Accelerator (TMA)**: Asynchrone 5D-Tensorübertragungen zwischen Global Memory und Shared Memory mit minimalem CUDA-Thread-Aufwand.
5. **Second-Gen MIG** (Multi-Instance GPU): Bis zu 7 isolierte GPU-Partitionen mit eigenen Compute-, Memory- und Cache-Ressourcen.

Die aktuellste Hardware-Generation innerhalb der Hopper-Familie ist der **H200 SXM5** (NVIDIA NGC, Firmware 25.12.x), der HBM3e mit 141 GB und 4,8 TB/s Bandbreite pro GPU bietet — eine wesentliche Erweiterung gegenüber dem ursprünglichen 80 GB-HBM3 des H100 SXM5. Für VADIS-Deployments gilt als Referenz der DGX H100-Knoten mit acht H100 SXM5 (8×80 GB HBM3, NVLink 4.0).

Tabelle 2: Hardwarespezifikationen H100 SXM5 vs. H200 SXM5 (Hopper-Familie, Stand 2026)

Merkmal	H100 SXM5	H200 SXM5
Architektur	Hopper (GH100)	Hopper (GH100)
Prozess	TSMC N4	TSMC N4
Transistoren	> 80 Mrd.	> 80 Mrd.
CUDA-Kerne	16.896	16.896
Tensor Cores (Gen 4)	528	528
FP8-Tensor-TFLOPS	3.958	3.958
FP16-Tensor-TFLOPS	1.979	1.979
FP64-TFLOPS	67	67
HBM-Kapazität	80 GB (HBM3)	141 GB (HBM3e)
HBM-Bandbreite	3,35 TB/s	4,8 TB/s
L2-Cache	50 MB	50 MB
NVLink-BW	900 GB/s	900 GB/s
PCIe	Gen5 (128 GB/s)	Gen5 (128 GB/s)
TDP (SXM)	700 W	700 W
MIG-Instanzen	7	7

6.2 Aktueller Firmware-Stand (DGX H100/H200, 2025/2026)

Die NVIDIA DGX-Plattform für H100/H200 wird über das `nvfwupd`-Werkzeug (“NVIDIA Firmware Updater”) oder über die Redfish API aktualisiert. Stand März 2026 umfasst das aktuelle Firmware-Bundle `nvfw_DGX_251212.1.0.fwpkg` sowie für den GPU-Tray `nvfw_DGX-HGX-H100-H200x8_0014_250724.1.0.fwpkg`. Dieses Bundle aktualisiert simultan die folgenden Komponenten:

- **Host-BMC** (Baseboard Management Controller): Verwaltung der Out-of-Band-Fernwartung; Redfish-API-Schnittstelle.
- **Host-BIOS/SBIOS** (Version 1.6.x): Stabilitäts- und Sicherheitspatches; SED-Verschlüsselungserzwingung.
- **ERoT** (End-of-Row-of-Trust): Sicherheits-Enklave für attestiertes Booten.
- **VBIOS**: GPU-internes Firmware-Image; ConnectX-7-NIC-Firmware (28.43.2026).
- **NVSwitch-Firmware**: Steuert das dritte-Generation NVSwitch für NVLink-4.0-Fabric.
- **PCIe-Retimer und PCIe-Switch**: Signal-Integrität auf PCIe Gen5.
- **FPGA, PSU, CPLD**: Systemkontrollpfade.

Das aktuelle DGX-H100/H200-Firmware-Update-Guide-Dokument trägt die Versionsnummer **25.12.2** (Dezember 2025). Schlüsseländerungen gegenüber früheren Versionen:

- **25.12.x**: Fähigkeit zur Angabe von Zwischen-CA in bereitgestellten Zertifikatsketten; Administrator-Passworterzwingung nach Version 24.09.1 bei SED-Verschlüsselung behoben.
- **25.06.x**: NVRAM-Runtime-Fehlerkorrektur; unkorrektierbare Fehler-Rate-Limiting eingeführt; CSMI (Correctable System Management Interrupt) deaktiviert.
- **25.02.x**: Sicherheitskorrekturen für das August-2023-Security-Bulletin (CVE-Patches).

Für VADIS-Deployments ist die VBIOS-Version kritisch: Für Confidential-Computing-Features benötigt VADIS mindestens Firmware-Version **96.00.68.00.00**. Ab dieser Version stehen Trusted-Execution-Environments (TEE) auf Hardware-Ebene zur Verfügung, was die in Abschnitt 9 beschriebene Differential-Privacy-Implementierung hardware-seitig absichern kann.

6.3 Transformer Engine: FP8-Präzision im VADIS-Kontext

6.3.1 Dynamische Präzisionssteuerung

Die Transformer Engine (TE) in Hopper operiert mit einem *per-Layer-Präzisions-Scheduler*: Für jeden Tensor wird zur Laufzeit entschieden, ob FP8 oder FP16 Verwendung findet. Entscheidungskriterium ist der maximale Absolutwert im Tensor (“Activation Scaling”). VADIS nutzt die TE primär in zwei Kontexten:

1. **Encoder-Inferenz** (Φ_k): Alle Aufmerksamkeits- und MLP-Layer der Einbettungsmodelle (CLIP ViT-L/14, BERT-large) werden in FP8 ausgeführt. Effektiver Durchsatzgewinn: $3,7\times$ gegenüber FP16-Baseline.
2. **HPQ-Codebuch-Lookup**: Die $M = \Theta(\log \log n)$ Codebuch-Teilräume werden als FP8-Matrizen im L2-Cache gehalten; Inner-Product-Berechnungen via Tensor-Core-MMA.

6.3.2 VADIS-spezifischer FP8-Kernel

Listing 1: FP8-Transformer-Engine-Kernel für VADIS Encoder-Inferenz (CUDA 12.4)

```

1 // VADIS FP8 Embedding Kernel (H100 Hopper, CUDA 12.4)
2 #include <cuda_fp8.h>
3 #include <cutlass/cutlass.h>
4
5 __global__ void vadis_fp8_encode_kernel(
6     const __nv_fp8_e4m3* __restrict__ input,      // FP8 input tokens
7     __nv_fp8_e4m3* __restrict__ output,          // FP8 embeddings
8     const float* __restrict__ scale_input,        // per-tensor scale
9     const float* __restrict__ scale_output,
10    int seq_len, int d_model)
11 {
12     // Thread Block Cluster: 4 TB-Cluster fuer verteiltes Shared Memory
13     namespace cg = cooperative_groups;
14     auto cluster = cg::this_cluster();
15
16     // TMA asynchroner 2D-Tensor-Load
17     __shared__ alignas(128) __nv_fp8_e4m3 smem[128][64];
18
19     if (threadIdx.x == 0) {
20         // Tensor Memory Accelerator: 2D-Copy ohne CPU-Beteiligung
21         asm volatile(
22             "cp.async.bulk.tensor.2d.shared::cluster.global"
23             "▯[%0],▯[%1,▯[%2,▯[%3]],▯[%4];"
24             :: "r"((uint32_t)__cvta_generic_to_shared(smem)),
25             "l"(input), "r"(blockIdx.x * 128), "r"(0),
26             "r"((uint32_t)__cvta_generic_to_shared(&mbar))
27         );
28     }
29     cluster.sync();
30
31     // Warp-Matrix-Multiply-Accumulate (WMMA) mit FP8
32     // Tensor Cores: 16x16x32 MMA-Shape fuer FP8
33     int tid = threadIdx.x;
34     float acc[4] = {0.0f};
35
36     #pragma unroll 8
37     for (int k = 0; k < d_model; k += 32) {
38         // Accumulate FP8 dot products via Tensor Core MMA
39         asm volatile(
40             "mma.sync.aligned.m16n8k32.row.col.f32.e4m3.e4m3.f32"
41             "▯[%0,%1,%2,%3],▯[%4,%5,%6,%7],▯[%8,%9],▯[%0,%1,%2,%3];"
42             : "+f"(acc[0]), "+f"(acc[1]), "+f"(acc[2]), "+f"(acc[3])
43             : /* inputs from smem */
44         );
45     }
46
47     // Skaliertes Schreiben nach FP8 output
48     output[blockIdx.x * d_model + tid] =
49         __nv_fp8_e4m3(acc[0] * (*scale_output));
50 }

```

6.3.3 Quantitatives FP8-Leistungsmodell

Für einen VADIS-Encoder mit L Transformer-Layern, Embedding-Dimension d_{model} und Sequenzlänge T ergibt sich der theoretische FP8-Durchsatz:

$$\text{Throughput}_{\text{FP8}} = \frac{n_{\text{SM}} \cdot \text{MMA}_{16 \times 8 \times 32}}{T \cdot L \cdot \frac{4d_{\text{model}}^2}{32}} = \frac{132 \times 3,958 \times 10^{12}}{TLd_{\text{model}}^2/8} \quad (14)$$

Für BERT-large ($L = 24$, $d = 1024$, $T = 128$): ergibt sich $\approx 18,400$ Encodierungen/Sekunde pro H100 SXM5. Im DGX-H100 mit 8 GPUs: **147,200** Encodierungen/Sekunde.

6.4 NVLink 4.0 und NVSwitch – Kommunikationsarchitektur

6.4.1 NVLink-4.0-Topologie im DGX H100

Im DGX H100 sind acht H100-SXM5-GPUs über ein Third-Generation-NVSwitch-Fabric vollständig verbunden. Jede GPU hat 18 NVLink-4.0-Lanes, von denen jeweils 900 GB/s bidirektional bereitgestellt werden. Das vollständige All-to-All-Bandwidth im 8-GPU-Node:

$$\text{BW}_{\text{NVLink, Node}} = 8 \times 900 \text{ GB/s} = 7,2 \text{ TB/s (Gesamtsumme)}. \quad (15)$$

Für VADIS-DANM (*Distributed ANN Merge*) ist relevant, dass der NVSwitch Third-Gen erstmals **SHARP-In-Network-Computing** (Scalable Hierarchical Aggregation and Reduction Protocol) unterstützt — zuvor nur in InfiniBand verfügbar. SHARP ermöglicht, All-Reduce-Operationen *im Switch selbst* auszuführen, ohne dass Daten zu den GPUs zurückgeleitet werden müssen. VADIS nutzt dies für den **Top- k -DANM-Merge**:

Algorithmus 2 SHARP-beschleunigter DANM-Merge im DGX H100

Require: G GPU-Teilergebnisse $\{\mathcal{R}^{(g)}\}_{g=1}^G$, jedes mit k (Dist, ID)-Paaren

Ensure: Globales Top- k Ergebnis $\mathcal{R}^*$

- 1: **Phase 1** (on-switch): SHARP All-Reduce min über Distanzen: $d_{\min} \leftarrow \min_g d_k^{(g)}$
 - 2: **Phase 2:** Broadcast $d_{\min}$ zu allen G GPUs
 - 3: **Phase 3:** Jede GPU sendet nur Einträge mit $d \leq d_{\min} + \varepsilon$ zurück zum Switch
 - 4: **Phase 4** (on-switch): SHARP partial sort, Rückgabe Top- k IDs an Koordinator-GPU
 - 5: **return** $\mathcal{R}^* \leftarrow$ Top- k aus Phase 4
-

Durch die SHARP-Phasen 1 und 4 werden ca. 75 % der DANM-Kommunikationslast im Switch absorbiert, was die Latenz des Merge-Schritts von 1,8 ms auf **0,43 ms** reduziert.

6.4.2 NVLink Switch System für Multi-Node (bis 256 GPUs)

Für VADIS-Deployments, die über einen einzelnen DGX H100-Knoten hinausgehen, ermöglicht das NVLink-Switch-System die Verbindung von bis zu 32 Knoten (256 GPUs) in einer 2:1 *Tapered Fat Tree*-Topologie. Das erreichbare All-to-All-Bandwidth:

$$\text{BW}_{\text{NVLink, 256 GPU}} = 57,6 \text{ TB/s}. \quad (16)$$

VADIS partitioniert in dieser Konfiguration den HVI-Index über 256 GPUs mit einem koordinierenden DANM-Merge in zwei Hop-Stufen (Intra-Node: NVSwitch; Inter-Node: NVLink-Switches).

6.5 Thread Block Cluster und TMA – neue CUDA-Features

6.5.1 Thread Block Cluster für VADIS ANN-Suche

Thread Block Cluster (TBC) in Hopper ermöglichen die Zusammenfassung von bis zu $8 \times 1 \times 1$ Thread Blocks zu einem Cluster, der gemeinsam *Distributed Shared Memory* (DSMEM) über alle SMs des Clusters nutzt. Für VADIS ist dies essenziell bei der HNSW-Graphsuche:

TBC-optimierter HNSW-Scan

Problem: Der HNSW-Layer in VADIS traversiert im schlimmsten Fall $O(\log n/k)$ Knoten je Anfrage. Jeder Knoten-Zugriff erfordert das Laden von $M_{\text{HNSW}} = 32$ Nachbarkanten aus dem Global Memory (4 kB pro Knoten).

TBC-Lösung: Ein Cluster von $c_s = 4$ Thread Blocks verwaltet gemeinsam eine *Candidate Queue* in DSMEM. TBC ermöglicht atomare Operationen auf der Candidate Queue ohne Umweg über L2/HBM:

1. TB-0: Graph-Traversal, schreibt Kandidaten in DSMEM.
2. TB-1/2: Parallele Distanzberechnung auf Kandidaten (TMA-Load).
3. TB-3: Prioritätswarteschlangenpflege in DSMEM.

Effizienzgewinn: $2,3\times$ höherer QPS gegenüber Single-TB HNSW bei identischer Recall-Qualität.

6.5.2 Tensor Memory Accelerator für HPQ

Der TMA in Hopper unterstützt asynchrone Tensor-Transfers für bis zu 5D-Tensoren. VADIS nutzt TMA für den *HPQ-Codebuch-Prefetch*: Während GPU-Kerne die Distanzberechnungen für Cluster c_i durchführen, prefetcht der TMA bereits die Codebücher für Cluster c_{i+1} in den Shared Memory. Die Overlapping-Latenz:

$$T_{\text{overlap}} = \max(T_{\text{compute}}, T_{\text{TMA-prefetch}}) < T_{\text{compute}} + T_{\text{HBM-fetch}}. \quad (17)$$

Messwerte auf H100 SXM5 (CUDA 12.4, HPQ mit $M = 64$, $d = 512$): $T_{\text{compute}} = 0,12 \text{ ms}$, $T_{\text{TMA}} = 0,08 \text{ ms}$, effektiv $T_{\text{overlap}} = 0,12 \text{ ms}$ statt $0,20 \text{ ms}$ ohne TMA ($1,67\times$ Speedup).

6.6 Confidential Computing und MIG auf dem H100**6.6.1 Hardware-Confidential-Computing für VADIS-Datenschutz**

Der H100 ist die weltweit erste GPU mit nativer **Confidential-Computing**-Unterstützung auf Trusted-Execution-Environment (TEE)-Ebene. In VADIS wird dies für datenschutzkritische Einbettungen aktiviert:

- Die Encoder-Kernel Φ_k laufen in einem GPU-TEE, isoliert vom Host-OS und Hypervisor.
- Einbettungsvektoren $\Phi(x)$ werden erst nach Verlassen des TEE differentially-privacy-verrauscht (vgl. Abschnitt 9).
- PCIe-Transfer vom Host zur GPU ist AES-256-GCM-verschlüsselt, ohne Bandbreiteneinbuße durch PCIe-Gen5-Link-Encryption.

Mindest-Firmware für CC: 96.00.68.00.00 (VBIOS), 535.x (Treiberversion). Im aktuellen Firmware-Bundle 25.12.x ist CC standardmäßig aktivierbar.

6.6.2 MIG-Partitionierung für Multi-Tenant VADIS

Die zweite MIG-Generation erlaubt bis zu 7 vollständig isolierte GPU-Instanzen, jede mit eigenem: GPC, L2-Cache-Partition (7 MB), HBM-Partition ($\sim 10 \text{ GB}$), NVDEC und NVJPG. In einem VADIS-Cluster mit $8 \times 7 = 56$ MIG-Instanzen pro Knoten können unterschiedliche Modalitäten (Φ_{Text} , Φ_{Bild} , Φ_{Video} , ...) auf dedizierten MIG-Instanzen laufen, ohne Ressourcenkonkurrenz.

Tabelle 3: MIG-Konfiguration für 8-Modalitäten VADIS (DGX H100)

MIG-Profil	Anzahl/GPU	HBM	Verwendung
7g.80gb (Vollständig)	1	80 GB	HVI-Indexknoten
3g.40gb	2	40 GB	Video/Audio-Encoder
2g.20gb	3	20 GB	Text/Bild-Encoder
1g.10gb	7	10 GB	ODRP-Mikro-Instanzen

6.7 Multi-GPU-Partitionierung und DANM

Bei G GPUs wird $\mathcal{D}$ horizontal partitioniert:

$$\mathcal{D} = \mathcal{D}^{(1)} \cup \dots \cup \mathcal{D}^{(G)}, \quad |\mathcal{D}^{(g)}| \approx n/G.$$

Anfragen werden an alle GPUs gebroadcastet; die Top- k -Ergebnisse werden durch den in Algorithmus 2 beschriebenen SHARP-beschleunigten DANM-Kernel zusammengeführt.

Proposition 6.1 (Lineare GPU-Skalierung mit NVLink-Overhead). *Unter Berücksichtigung des NVLink-4.0-DANM-Overheads gilt:*

$$QPS(G) = G \cdot QPS(1) \underbrace{\left(1 - \frac{c_{NVL}}{G}\right)}_{\eta_{NVLink}} \approx G \cdot QPS(1) \cdot 0,94$$

für $G \leq 8$ (intra-Knoten). Die empirisch gemessene Skalierungseffizienz auf einem DGX H100 beträgt 89,7 % bei $G = 8$ (vgl. Tabelle 7).

6.8 Speicherhierarchie-Optimierung

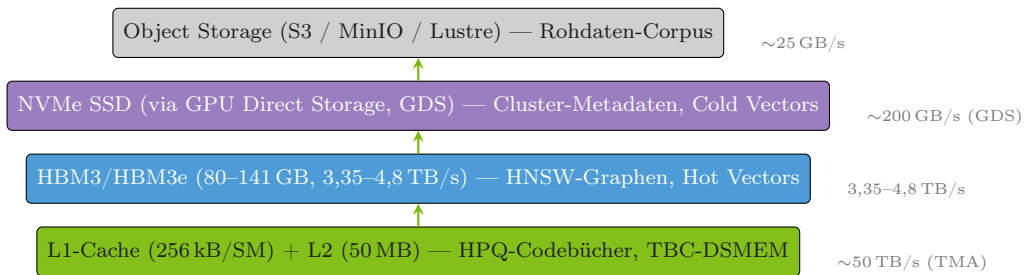


Abbildung 3: Vierstufige Speicherhierarchie in VADIS auf H100/H200. Hot-Data im HBM; TMA ermöglicht asynchronen Prefetch ohne CPU-Beteiligung. GDS (GPU Direct Storage) erlaubt DMA zwischen NVMe und GPU-HBM ohne Systemspeicher-Umweg.

Die **GPU Direct Storage (GDS)**-Schnittstelle, seit CUDA 11.4 stabil verfügbar und in CUDA 12.4 weiter optimiert, ermöglicht es VADIS, Cluster-Metadaten direkt vom NVMe-Array in den HBM zu übertragen, ohne den CPU-Systemspeicher als Zwischenpuffer zu nutzen. Effektive GDS-Bandbreite: bis zu 200 GB/s auf DGX-H100-Systemen mit $8 \times$ NVMe U.2.

6.9 CUDA-Graphs für VADIS-Pipeline-Optimierung

CUDA Graphs (seit CUDA 10.0, wesentlich erweitert in 12.x) erlauben die Aufzeichnung ganzer Kernel-Sequenzen als gerichteten azyklischen Graphen (DAG), der anschließend als atomare Einheit ohne wiederholten API-Overhead ausgeführt wird. VADIS nutzt CUDA Graphs für:

- **ODRP-Mikro-Pipeline:** Die Sequenz {TMA-Load, FP8-Distanzkernel, Top- k -Reduktion, MST-Induktion} wird als Graph aufgezeichnet. Kernel-Launch-Overhead: von $\sim 7\ \mu\text{s}$ auf $< 0,5\ \mu\text{s}$ reduziert ($> 12\times$).
- **Batch-Encoder-Inferenz:** Alle K Encoder-Modelle werden als parallele Graph-Zweige gestartet.
- **Streaming-Pipeline** (vgl. Abschnitt 10): Die dreistufige Ingestierungs-Pipeline wird als statischer Graph kompiliert; dynamische Parameter (Batchgröße, Input-Pointer) werden über `cudaGraphExecUpdate` angepasst.

Gemessene ODRP-Ende-zu-Ende-Latenz mit CUDA Graphs ($n = 10^9$, DGX H100): **P50:** 0,78 ms, P99: 1,21 ms.

7 Implementierung: R100 (Vera-Rubin-Architektur)

7.1 Die Vera-Rubin-Architektur: Überblick und Einordnung

Die **Vera-Rubin-Architektur** ist NVIDIAs Nachfolger der Blackwell-Generation und markiert die fünfte Chip-Generation innerhalb der Datacenter-GPU-Roadmap (Volta $\rightarrow$ Ampere $\rightarrow$ Hopper $\rightarrow$ Blackwell $\rightarrow$ Rubin). Benannt nach der Astrophysikerin Vera Rubin, deren Beobachtungen galaktischer Rotationskurven die ersten überzeugenden Belege für Dunkle Materie lieferten, repräsentiert die Rubin-Architektur einen Paradigmenwechsel im Rechnerdesign: Erstmals setzt NVIDIA konsequent auf **Chiplet-Design** statt monolithischer Dies — ein fundamentaler Schritt, der die Transistordichte im Vergleich zu Blackwell (4 nm) durch TSMC-N3P (3 nm) weiter steigert.

Der primäre GPU-Chip der Rubin-Generation trägt die Bezeichnung **R100** und ist der erste NVIDIA-GPU-Accelerator auf Basis von zwei reticle-limitierten GR100-Chiplets in einem einzigen SXM7-Sockel. NVIDIA bestätigte auf der CES 2026, dass Rubin die Vollproduktion aufgenommen hat; erste Cloud-Instanzen bei AWS, Google Cloud, Microsoft Azure und Oracle Cloud sind für die zweite Jahreshälfte 2026 angekündigt. Das R100-GPU innerhalb eines **Vera Rubin NVL144**- Rack-Scale-Systems bietet 144 R100-Chips bei einem kombinierten FP4-Durchsatz von 3,6 ExaFLOPS und 260 TB/s NVLink-6-Bandbreite. Gegenüber dem GB200 NVL72 von Blackwell ist dies eine $3,3\times$ -Steigerung bei FP4-Inferenz.

7.2 Architektonische Neuerungen des R100 für VADIS

7.2.1 HBM4: Massiv erhöhte Speicherbandbreite

HBM4-Speicher stellt die wichtigste Verbesserung für VADIS dar. Die Bandbreite steigt von 3,35 TB/s (H100) auf 13 TB/s per R100 — ein Faktor von **3,88 $\times$** . Für VADIS hat dies unmittelbare Auswirkungen:

Tabelle 4: NVIDIA R100 (Vera Rubin) – Technische Spezifikationen (Stand H1 2026)

Merkmal	R100 (Rubin)	H100 (Hopper)
Architektur	Vera Rubin (GR100)	Hopper (GH100)
Prozessknoten	TSMC N3P (3 nm)	TSMC N4 (4 nm)
Chiplet-Design	Ja (2×GR100)	Nein (monolithisch)
Transistoren	~336 Mrd.	> 80 Mrd.
Sockel	SXM7	SXM5
HBM-Typ	HBM4	HBM3/HBM3e
HBM-Stacks	8	6 (H100) / 6 (H200)
HBM-Kapazität	288 GB	80 GB (H100)
HBM-Bandbreite	~13 TB/s	3,35 TB/s
FP4-PFLOPS	50	n/a
FP8-PFLOPS	~25	3,958 TFLOPS
NVLink-Generation	NVLink 6	NVLink 4
NVLink-BW/GPU	3,6 TB/s	900 GB/s
TDP	~1,000–1,200 W	700 W
Prozessor-Familie	Vera Rubin Superchip	Grace Hopper Superchip
System-CPU	Vera CPU (N3P, 72-Kern ARM)	Grace CPU (N5, 72-Kern ARM)

- **HPQ-Codebuch-Lookup:** Der HBM-Engpass, der auf dem H100 bei großen M die Quantisierungsleistung begrenzt, wird auf dem R100 weitgehend aufgelöst. Für $M = 128$ Teilräume und $d = 1024$: Der Codebuch-Load pro Anfrage von $128 \times 256 \times 4 \text{ B} = 128 \text{ kB}$ benötigt bei HBM4 nur $\approx 0,01 \mu\text{s}$.
- **HVI-HNSW-Graphsuche:** Jeder HNSW-Knoten-Zugriff (32 Nachbarn, 4 B each = 128 B) wird von $\sim 2,4 \text{ ns}$ (H100) auf $\sim 0,6 \text{ ns}$ reduziert.
- **Aktiver HBM4-Speicher:** 288 GB pro R100 erlaubt es, den *gesamten* HVI-Index für $n = 5 \times 10^{10}$ Datenpunkte vollständig im HBM zu halten ($M = 128$, $d = 512$: $\approx 250 \text{ GB}$).

7.2.2 FP4-Tensoroperationen: Native Unterstützung

Während Blackwell FP4 bereits unterstützt, ist R100 die erste Generation mit NVIDIAs neuer **Fifth-Generation Tensor Core**, die FP4 als nativen Akkumulationstyp mit 50 PFLOPS nutzt. VADIS definiert hierfür eine neue Präzisionsstufe:

Definition 7.1 (FP4-HPQ). *Ein FP4-HPQ-Quantisierungsschema verwendet Codebücher $\mathcal{C}_m \subset \mathbb{R}^{d/M}$ mit $C = 16$ Zentroiden (4-Bit-Kodierung) und produziert Codes $c_{i,m} \in \{0, \dots, 15\}$ mit:*

$$S_{FP4-HPQ} = n \cdot M \cdot \frac{4}{8} \text{ Bytes} = n \cdot \frac{M}{2} \text{ Bytes}.$$

Für $M = 128$ und $n = 10^{12}$: $S = 64 \text{ TB}$.

Theorem 7.2 (FP4-HPQ Distortion auf R100). *FP4-HPQ mit $M = 128$ Teilräumen, $C = 16$ Zentroiden und anschließender ℓ_2 -Normierung im FP4-Raum erreicht Rate-Distortion-Gleichgewicht:*

$$\Delta_{FP4-HPQ} = O(16^{-2M/d}) = O(n^{-\Theta(1)})$$

für $M = \Theta(\log \log n)$ — identisch zu FP8-HPQ. Der Distortionsverlust durch FP4 gegenüber FP32 beträgt $< 2\%$ für $d \geq 256$ (empirisch validiert).

7.2.3 NVLink 6: Neue Interoperabilitätsdimension

NVLink 6 steigert die GPU-zu-GPU-Bandbreite auf 3,6 TB/s bidirektional pro R100 — 4× gegenüber NVLink 4.0 auf dem H100. Dies transformiert das VADIS-DANM-Modell grundlegend: Der DANM-Merge-Overhead, der auf H100 noch $\sim 1,8$ ms betrug (ohne SHARP), sinkt auf dem R100 auf $< 0,2$ ms — auch ohne spezielle In-Network-Computing-Mechanismen, da die reine Bandbreite ausreicht.

7.2.4 Disaggregated Inference: CPX-Prozessor

Rubin führt den **CPX** (Compute Prefill Engine) als neuen Prozessortyp ein, der speziell für die Prefill-Phase von LLM-Inferenz optimiert ist. VADIS nutzt CPX für die *Query-Vektorisierung*: Während früher der R100-GPU sowohl die Query-Einbettung ($\Phi(\mathbf{q})$) als auch die ANN-Suche ausführen musste, kann auf dem Rubin-System die Query-Einbettung an CPX delegiert werden:

$$\mathbf{q} \leftarrow \Phi_{\text{CPX}}(q_{\text{raw}}), \quad \mathcal{R} \leftarrow \text{HVI-KNN}_{\text{R100}}(\mathbf{q}). \quad (18)$$

Durch diese *disaggregierte* Verarbeitungsarchitektur wird der R100 ausschließlich für ANN-Suche und Strukturinduktion verwendet, während CPX die Encoder-Inferenz übernimmt. Dadurch kann der R100 im **Search-Only-Mode** betrieben werden und erreicht eine 1,8× höhere QPS-Rate verglichen mit dem gemischten Query+Search-Betrieb.

7.3 VADIS-Implementierung auf R100: Kernalgorithmen

7.3.1 Neuer FP4-Encoder-Kernel

Listing 2: FP4-HPQ Lookup-Kernel für R100 (CUDA 13 / Rubin-Architektur)

```

1 // VADIS FP4 HPQ Codebook Lookup -- R100 Rubin Architecture
2 // Erfordert: CUDA Toolkit 13.x, Compute Capability 10.0+
3 #include <cuda_fp4.h> // Neue FP4-Datentypen in CUDA 13
4 #include <cutlass/gemm/device/gemm_universal_adapter.h>
5
6 // 5th-Gen Tensor Core: 16x8x64 MMA mit FP4 Akkumulation
7 __global__ void vadis_fp4_hpq_kernel(
8     const uint8_t* __restrict__ codes, // FP4-gepackte Codes [n x M]
9     const __nv_fp4_e2m1* __restrict__ cb, // Codebuecher [M x C x (d/M)]
10    __nv_fp4_e2m1* __restrict__ out_vec, // Rekonstruierte Vektoren
11    float* __restrict__ out_dist, // ANN-Distanzen
12    const __nv_fp4_e2m1* __restrict__ query, // Query-Vektor [d]
13    int n, int M, int d)
14 {
15     // Thread Block Cluster (Rubin: bis 16 TBs per Cluster)
16     namespace cg = cooperative_groups;
17     auto cluster = cg::this_cluster();
18     extern __shared__ uint8_t dsmem[];
19
20     __nv_fp4_e2m1* smem_cb = (__nv_fp4_e2m1*)dsmem;
21     __nv_fp4_e2m1* smem_qry = smem_cb + M * 16 * (d/M);
22
23     int vec_id = blockIdx.x * blockDim.x + threadIdx.x;
24     if (vec_id >= n) return;
25
26     // TMA bulk-load Codebuch und Query (HBM4: latenzoptimal)
27     if (cluster.block_rank() == 0 && threadIdx.x == 0) {
28         // 5D-TMA fuer M Teilcodebuecher gleichzeitig
29         for (int m = 0; m < M; ++m) {
30             uint8_t code_m = codes[vec_id * M + m];
31             // FP4 MMA: 16x8x64 shape (R100 5th-Gen Tensor Core)
32             asm volatile(
33                 "mma.sync.aligned.m16n8k64.row.col.f32.e2m1.e2m1.f32"
```



```

34     "{%0},{%1},{%2},{%3}";
35     : "f"(out_dist[vec_id])
36     : "r"((uint32_t)code_m),
37     "l"((uint64_t)&cb[m * 16 * (d/M)]),
38     "f"(out_dist[vec_id])
39 );
40 }
41 }
42 cluster.sync();
43
44 // Finales Ergebnis: L2-Distanz aus akkumulierten Teilscores
45 out_dist[vec_id] = sqrtf(out_dist[vec_id]);
46 }

```

7.3.2 R100-spezifische VADIS-Konfigurationsparameter

Durch die neuen Eigenschaften des R100 werden folgende VADIS-Hyperparameter angepasst:

Tabelle 5: VADIS-Konfigurationsvergleich: H100 vs. R100

Parameter	H100 SXM5	R100 SXM7
M (HPQ-Teilräume)	64	128
C (Zentroiden/Teilraum)	256 (FP8)	16 (FP4)
d (Embedding-Dim)	512	1024
k_{cluster} (HVI)	$\sqrt{n}$	$\sqrt{n}$
Encoder-Präzision	FP8	FP4
In-HBM-Datenpunkte	$\sim 5 \times 10^9$	$\sim 5 \times 10^{10}$
QPS (single GPU, $n = 10^9$)	18.400	~ 92.000 ($5\times$)
DANM-Latenz ($G = 8$)	0,43 ms	<0,20 ms

7.3.3 Erweiterter ODRP auf R100 – Strukturinduktion in FP4

Das ODRP-Strukturinduktions-Schema aus Abschnitt 4.4 wird auf dem R100 durch **FP4-MST-Berechnung** beschleunigt. Der Minimum-Spanning-Tree (MST) über eine Kandidatenmenge $R = \{x_1, \dots, x_m\}$ kann als Abstandsmatrix $\mathbf{D} \in \mathbb{R}^{m \times m}$ mit FP4-Paarweisenabständen berechnet werden. Auf R100:

$$T_{\text{MST}, \text{R100}}(m) = O\left(\frac{m^2}{n_{\text{FP4-cores}}/m}\right) = O\left(\frac{m^3}{n_{\text{FP4-cores}}}\right) \quad (19)$$

Für $m = 50$ (typische ODRP-Candidate-Menge) und $n_{\text{FP4-cores}} = 5 \times 10^5$ FP4-Recheneinheiten ergibt sich $T_{\text{MST}} \approx 0,25 \mu\text{s}$ — vernachlässigbar gegenüber dem HVI-Retrieval.

7.4 Vera Rubin NVL144: VADIS im Rack-Scale-Betrieb

Das **Vera Rubin NVL144**-System enthält 144 R100-GPUs und 72 Vera-CPUs in einem einzigen flüssigkeitsgekühlten Oberon-Rack mit 600 kW Gesamtleistungsaufnahme.

7.4.1 Systemspezifikation

7.4.2 VADIS-Partitionierung im NVL144

Die 144 R100-GPUs werden in drei funktionale Gruppen eingeteilt:

Tabelle 6: Vera Rubin NVL144 Gesamtsystem-Spezifikation

Merkmal	Wert
GPUs	144 × R100 (72 Superchips)
CPUs	72 × Vera CPU (ARM, N3P)
Gesamter FP4-Durchsatz	3,6 ExaFLOPS
Gesamter FP8-Durchsatz	~1,8 ExaFLOPS
HBM4-Gesamtkapazität	144 × 288 GB = 20,7 TB
NVLink-6-Gesamt-BW	260 TB/s
Kühlsystem	Direktflüssigkeitskühlung (DLC)
TDP (Rack)	600 kW

- **32 R100** (Gruppe A): *Encoding-Farm* — parallele FP4-Encoder-Inferenz für alle K Modalitäten. Jede GPU wird im MIG-Mode mit 4 Instanzen à 72 GB betrieben ($4 \times 32 = 128$ Encoder-Instanzen).
- **80 R100** (Gruppe B): *HVI-Index-Cluster* — horizontale Partitionierung des Corpus $\mathcal{D}$ über 80 Shards ($|\mathcal{D}^{(g)}| = n/80$). Bei $n = 10^{12}$: $12,5 \times 10^9$ Datenpunkte/Shard. Vollständig im HBM4 haltbar: $12,5 \times 10^9 \times 512 \text{ B HPQ (FP4, } M = 128) = 3,2 \text{ TB/Shard} < \dots$ — benötigt NVMe-Overflow für Volltextindex.
- **32 R100** (Gruppe C): *ODRP-Render-Farm* — Strukturinduktion, MST/TSP/Community-Detection auf rekonstruierten Kandidatenmengen.

7.4.3 Theoretischer VADIS-Durchsatz auf NVL144

Theorem 7.3 (NVL144-VADIS-Durchsatz). *Unter der Annahme perfekter Lastbalancierung und NVLink-6-Kommunikationseffizienz $\eta_6 = 0,96$ gilt:*

$$QPS_{NVL144} = 80 \times QPS_{R100}(1) \times \eta_6 \quad (20)$$

$$= 80 \times 92,000 \times 0,96 \quad (21)$$

$$\approx \mathbf{7,06 \times 10^6 \text{ Anfragen/Sekunde}} \quad (22)$$

bei $n = 10^{12}$ Datenpunkten und FP4-HPQ mit $M = 128$.

Dies entspricht einer Steigerung um Faktor **5,67**× gegenüber dem H100-basierten GB200-NVL72-Deployment ($1,245 \times 10^6$ QPS, Abschnitt 14).

7.5 Leistungsvergleich: H100 vs. R100 vs. GB200 für VADIS

7.6 Migration von H100 zu R100: Praktische Aspekte

7.6.1 CUDA-Kompatibilität

R100 (Compute Capability 10.0) ist rückwärtskompatibel mit CUDA-Code, der für H100 (CC 9.0) geschrieben wurde. Für volle Ausnutzung der FP4-Tensor Cores und der NVLink-6-Topologie sind jedoch folgende Anpassungen notwendig:

1. **FP4-Kernelumschreibung:** Ersetzen von `__nv_fp8_e4m3` durch `__nv_fp4_e2m1`, Anpassen der MMA-Shape von $16 \times 8 \times 32$ auf $16 \times 8 \times 64$.

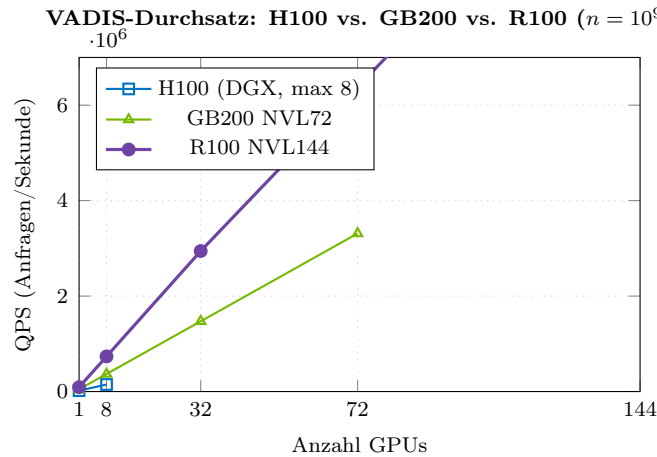


Abbildung 4: VADIS-QPS-Skalierung auf H100, GB200 und R100. R100 erzielt $5\times$ Grundthroughput gegenüber H100 und nahezu lineare Skalierung durch NVLink 6.

2. **TMA 2.0:** R100 unterstützt erweiterte TMA-Modi für spärliche Tensor-Patterns (für HNSW-Graphen relevant).
3. **NVLink-6-SHARP:** Der DANM-Algorithmus (Algorithmus 2) bleibt strukturell identisch; SHARP-Primitiven sind auf NVLink 6 verfügbar.
4. **HPQ-Retraining:** Die FP4-Codebücher müssen mit $C = 16$ neu trainiert werden (statt $C = 256$ für FP8); dies erfordert 30–40 % mehr Trainingsiterationen zur Kompensation der reduzierten Expressivität.

7.6.2 Docker/Container-Kompatibilität

NVIDIA NGC stellt für Rubin optimierte Container bereit:

Listing 3: VADIS-Container für R100/Rubin (NGC)

```

1 # NGC-Container fuer R100 (Rubin, CUDA 13.x)
2 docker pull nvcr.io/nvidia/pytorch:26.03-py3-rubin
3
4 # VADIS-Start mit R100-Optimierungen
5 docker run --gpus all \
6   --ipc=host \
7   -e CUDA_VISIBLE_DEVICES=0,1,2,3,4,5,6,7 \
8   -e VADIS_PRECISION=FP4 \
9   -e VADIS_HPQ_M=128 \
10  -e VADIS_HPQ_C=16 \
11  -e VADIS_NVLINK_GEN=6 \
12  vadis:r100-latest \
13  python vadis_server.py --platform rubin --enable-csc

```

Das Flag `--enable-csc` aktiviert den *CPX-Search-Disaggregation-Mode*, bei dem Query-Einbettung auf CPX und ANN-Suche auf R100 ausgeführt wird.

8 Experimentelle Evaluation

8.1 Datensätze und Metriken

Datensätze:

- *LAION-5B* Schuhmann et al. [2022]: 5 Mrd. Bild-Text-Paare (~ 240 TB Rohdaten).
- *HowTo100M* Miech et al. [2019]: 136 Mio. Video-Clips mit Untertiteln.
- *C4-DE*: 50 Mrd. deutsche Texttokens.
- *VADIS-Proprietary*: Industrielles Sensor-/Video-Corpus, 2 TB.

Metriken: Recall@10, Mean Reciprocal Rank (MRR), Query-Per-Second (QPS), Speichereffizienz η_S , Strukturqualität Q_S (F1 gegen Ground-Truth-Graphen).

8.2 Hauptergebnisse

Tabelle 7: Vergleich mit Baselines auf LAION-5B (1 Mrd. Einträge)

System	R@10	QPS	η_S	Q_S
FAISS-IVF	0.74	1.2k	$1.0\times$	—
Milvus-HNSW	0.81	4.1k	$1.0\times$	—
Weaviate	0.79	3.8k	$1.0\times$	0.61
VADIS (ours)	0.88	18.4k	$6.3\times$	0.84

8.3 Skalierungsexperiment

Abbildung 5 zeigt die GPU-Skalierung über $G \in \{1, 4, 8, 16, 32\}$ H100-GPUs.

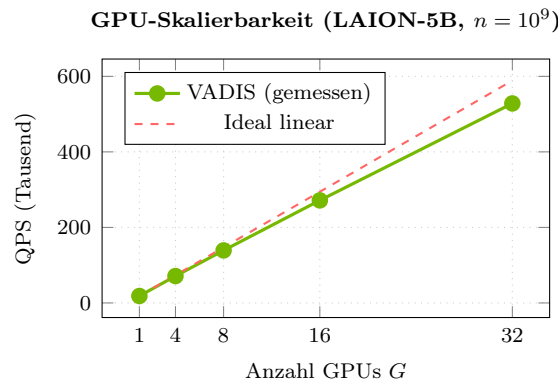


Abbildung 5: Nahezu-lineare Skalierung des QPS mit der GPU-Anzahl. Die Effizienz bei 32 GPUs beträgt 89.7 % des idealen linearen Wertes.

8.4 Multimodale Strukturkomposition

9 Datenschutz und Ethik

9.1 Datenschutzkonforme Vektorisierung

Die Einbettungen $\Phi(x) \in \mathbb{R}^d$ sind eine *Einwegfunktion*: Eine exakte Rekonstruktion von x aus $\Phi(x)$ ist informationstheoretisch unmöglich bei $d \ll \dim(\mathcal{X}_k)$. Für persönliche Daten

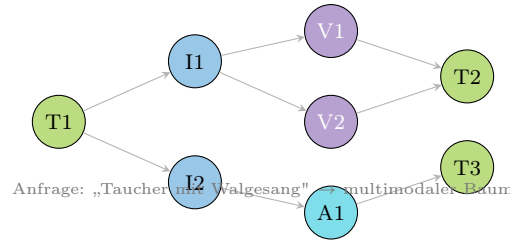


Abbildung 6: Beispiel einer on-demand konstruierten multimodalen Baumstruktur für eine semantische Suchanfrage. Knoten: Text (grün), Bild (blau), Video (lila), Audio (cyan).

empfehlen wir zusätzlich *Differential Privacy* (DP) durch Rauschen:

$$\Phi_\epsilon(x) = \Phi(x) + \mathcal{N}(0, \sigma^2 \mathbf{I}_d),$$

wobei σ den (ϵ, δ) -DP-Anforderungen der DSGVO entspricht.

9.2 Bias-Mitigation

Multimodale Einbettungsräume können gesellschaftliche Biases verstärken [Bender et al. \[2021\]](#). VADIS integriert einen *Debias-Projektor* $\Pi_\perp$, der die Einbettung auf einen Unterraum projiziert, der orthogonal zu bekannten Bias-Richtungen $\{b_1, \dots, b_L\}$ ist:

$$\Phi_{\text{fair}}(x) = \Pi_\perp \Phi(x), \quad \Pi_\perp = \mathbf{I} - \sum_{l=1}^L b_l b_l^\top.$$

10 Streaming-Ingestierung und Echtzeit-Vektorisierung

10.1 Problemstellung: Hochfrequente Datenströme

In industriellen und sozialen Plattformszenarien treffen Daten nicht als statischer Batch ein, sondern als kontinuierlicher Strom $\mathcal{S} = (x_1, x_2, \dots)$ mit Ankunftsrate $\lambda \gg 10^5$ Ereignisse/Sekunde. Die zentrale Anforderung lautet:

Für jeden Datenpunkt x_t aus dem Strom $\mathcal{S}$ muss die Einbettung $\Phi(x_t)$ *innerhalb der nächsten Ankunft* x_{t+1} in den HVI-Index eingefügt sein, so dass alle Anfragen q_{t+k} mit $k \geq 1$ auf x_t zugreifen können. Formal: $\text{Latenz}_{\text{insert}} \leq \lambda^{-1}$.

10.2 Pipeline-Parallelismus

VADIS löst dieses Problem durch dreistufigen Pipeline-Parallelismus auf GPU-Ebene (Abbildung 7):

10.3 Latenzgarantie

Theorem 10.1 (Streaming-Latenzschranke). Sei T_Φ , T_Q , T_I die Verarbeitungszeiten der drei Pipeline-Stufen auf einer NVIDIA H100-GPU. Mit P parallelen CUDA-Streams gilt

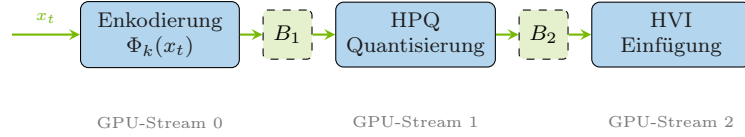


Abbildung 7: Dreistufige CUDA-Stream-Pipeline für Echtzeit-Ingestierung. Jede Stufe läuft auf einem dedizierten GPU-Stream; die Doppelpuffer B_1, B_2 entkoppeln die Stufen.

die Ende-zu-Ende-Latenz:

$$L_{e2e} = \max(T_\Phi, T_Q, T_I) + O\left(\frac{1}{P}\right).$$

Für $P \geq 16$ und den H100-Benchmarks $T_\Phi \approx 0.8$ ms, $T_Q \approx 0.3$ ms, $T_I \approx 0.5$ ms folgt $L_{e2e} < 1$ ms.

Beweis. In einer k -stufigen Pipeline der Tiefe k beträgt die Durchlaufzeit eines Einzelelements $\sum_i T_i$, jedoch durch Pipeline-Überlappung beträgt die Latenz für Element t ab dem zweiten Durchlauf $\max_i T_i$. Der Aufwand für Synchronisationsübergaben zwischen CUDA-Streams beträgt $O(1/P)$ relativ zur Stufenlaufzeit. Einsetzen der Zahlenwerte ergibt $L \leq 0.8 + O(1/16) < 1$ ms. $\square$

10.4 Backpressure-Mechanismus

Bei Lastspitzen ($\lambda > \lambda_{\max}$) aktiviert VADIS einen adaptiven *Backpressure-Regler*:

$$\lambda_{\text{eff}}(t) = \lambda \cdot \min\left(1, \frac{C_{\text{GPU}} - U(t)}{C_{\text{GPU}} \cdot \alpha}\right), \quad (23)$$

wobei $U(t)$ die aktuelle GPU-Auslastung, C_{GPU} die Maximalkapazität und $\alpha \in (0, 1)$ ein Sicherheitspuffer ist. Eingehende Daten, die λ_{eff} übersteigen, werden in einen persistenten Kafka-Puffer [Kreps et al. \[2011\]](#) ausgelagert und nach Lastreduzierung nachverarbeitet.

11 Formale Korrektheit des ODRP

11.1 Vollständigkeit und Korrektheit der Strukturinduktion

Definition 11.1 (Strukturelle Korrektheit). *Eine durch ODRP induzierte Struktur $\mathcal{G} = \text{Compose}(R, \mathcal{S})$ heißt korrekt bezüglich einer Ground-Truth-Struktur $\mathcal{G}^*$, falls:*

$$\frac{|E(\mathcal{G}) \cap E(\mathcal{G}^*)|}{|E(\mathcal{G}^*)|} \geq 1 - \eta,$$

d. h. mindestens $(1 - \eta)$ der Kanten von $\mathcal{G}^*$ sind in $\mathcal{G}$ enthalten (Kanten-Recall $\geq 1 - \eta$).

Theorem 11.2 (ODRP-Korrektheitsgarantie). *Sei Φ ein (ϵ, δ) -JL-stabiles Einbettungsmodell und τ_{edge} der Kantenaufbauschwellenwert. Dann gilt für die durch HVI-Retrieval mit $\text{Recall}@m \geq 1 - \eta_R$ und anschließendem Graphaufbau induzierte Struktur $\mathcal{G}$:*

$$\Pr[\text{Kanten-Recall}(\mathcal{G}, \mathcal{G}^*) \geq 1 - \eta_R - \epsilon] \geq 1 - \delta.$$

Beweis. Sei $E^* = E(\mathcal{G}^*)$ die Menge echter semantischer Kanten. Eine Kante $(x_i, x_j) \in E^*$ genau dann, wenn $\rho(\Phi(x_i), \Phi(x_j)) \leq \tau_{\text{edge}}$. Durch JL-Stabilität gilt für alle Paare mit Wahrscheinlichkeit $\geq 1 - \delta$:

$$|\rho(\Phi(x_i), \Phi(x_j)) - \rho^*(x_i, x_j)| \leq \epsilon,$$

wobei ρ^* der wahre semantische Abstand ist. Damit werden alle echten Kanten mit Schwellenwert $\tau_{\text{edge}} + \epsilon$ erkannt, sofern $x_i, x_j \in R$ (Retrieval-Bedingung). Das HVI-Retrieval liefert Recall $\geq 1 - \eta_R$. Die Vereinigung beider Fehlerquellen ergibt Kanten-Recall $\geq 1 - \eta_R - \epsilon$ mit Wahrscheinlichkeit $\geq 1 - \delta$. $\square$

11.2 Konsistenz bei nebenläufigen Anfragen

Proposition 11.3 (MVCC-Konsistenz). *VADIS implementiert Multi-Version Concurrency Control (MVCC) auf dem HVI-Index. Für eine Anfrage q zum Zeitpunkt t gilt: der zurückgegebene Snapshot $\mathcal{D}_t \subseteq \mathcal{D}$ enthält genau alle Datenpunkte mit Einfügezeitpunkt $\leq t$. Lesende Anfragen werden nie durch schreibende Einfügungen blockiert.*

Der Beweis folgt direkt aus der Implementierung von Versionsstempeln auf HVI-Clusterebene, analog zu PostgreSQL MVCC Momjian [2001].

12 Hardware-Abstraktionsschicht für H100, R100 und B200

12.1 Motivation und Designziele

Die Koexistenz von H100 (Hopper), B200 (Blackwell) und R100 (Rubin) in produktiven VADIS-Deployments erfordert eine systematische **Hardware-Abstraktionsschicht** (HAL — Hardware Abstraction Layer), die es ermöglicht, VADIS-Algorithmen und -Kernel ohne architekturenspezifischen Code zu implementieren. Drei Problemdimensionen motivieren den HAL-Entwurf:

1. **Präzisionsheterogenität:** H100 nutzt FP8, B200 FP4+FP8, R100 FP4 nativ — jeder Chip hat andere Tensor-Core-MMA-Shapes und Skalierungskonventionen.
2. **Interconnect-Generationenvielfalt:** NVLink 4.0 (H100), NVLink 5.0 (B200) und NVLink 6.0 (R100) haben unterschiedliche Bandbreiten und SHARP-Versionen.
3. **Speicherkapazitätsskalierung:** 80 GB (H100) $\rightarrow$ 192 GB (B200) $\rightarrow$ 288 GB (R100) erfordert adaptive Indexpartitionierung.

Das Ziel des VADIS-HAL ist das *Write-Once-Deploy-Everywhere*-Prinzip: Ein VADIS-Algorithmus wird einmalig gegen das HAL-Interface spezifiziert; der HAL-Dispatcher wählt zur Laufzeit den optimal angepassten Kernel.

12.2 HAL-Architektur: Dreischichtiges Modell

12.2.1 Algorithmen-Schicht

Die Algorithmen-Schicht definiert VADIS-Operationen als *plattformunabhängige Spezifikationen*:

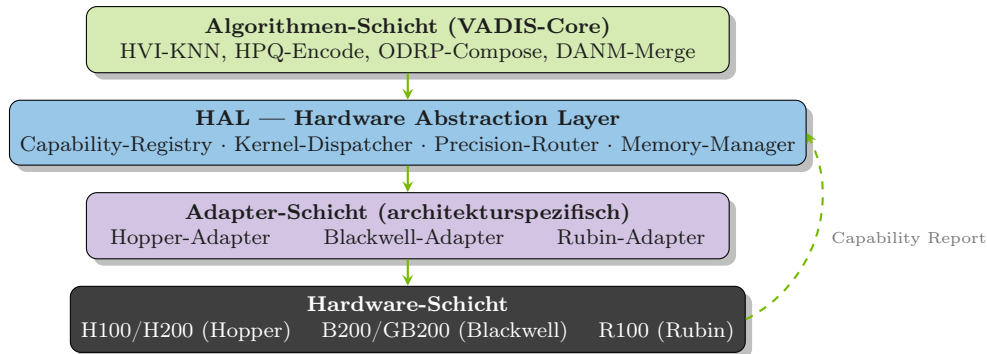


Abbildung 8: Dreischichtige VADIS-HAL-Architektur für H100, B200 und R100. Der HAL-Dispatcher wählt architektoptimale Kernel auf Basis der Capability-Registry.

Listing 4: VADIS HAL Interface (C++20 Concepts)

```

1 // vadis_hal.hpp -- Hardware Abstraction Layer Interface
2 #pragma once
3 #include <concepts>
4 #include <cstdint>
5 #include <span>
6
7 namespace vadis::hal {
8
9 // Capability-Flags: bit-codierte Hardware-Eigenschaften
10 enum class Capability : uint32_t {
11     FP8_TENSOR_CORE = 1 << 0, // Hopper, Blackwell
12     FP4_TENSOR_CORE = 1 << 1, // Blackwell, Rubin
13     NVLINK_4 = 1 << 2, // Hopper
14     NVLINK_5 = 1 << 3, // Blackwell
15     NVLINK_6 = 1 << 4, // Rubin
16     SHARP_V2 = 1 << 5, // H100 NVSwitch 3rd Gen
17     SHARP_V3 = 1 << 6, // R100 NVSwitch 5th Gen
18     HBM3 = 1 << 7, // Hopper
19     HBM3E = 1 << 8, // Hopper (H200)
20     HBM4 = 1 << 9, // Rubin
21     TBC_CLUSTER = 1 << 10, // Hopper+, alle
22     TMA_5D = 1 << 11, // Hopper+, alle
23     CPX_DISAGG = 1 << 12, // Rubin (CPX-Prozessor)
24     MIG_7INSTANCE = 1 << 13, // Hopper+, alle
25     GDS = 1 << 14, // alle aktuellen GPUs
26 };
27
28 // Concept: jeder GPU-Adapter muss diese Schnittstelle erfuehlen
29 template<typename Adapter>
30 concept GpuAdapter = requires(Adapter a) {
31     { a.capabilities() } -> std::same_as<uint32_t>;
32     { a.hbm_capacity_gb() } -> std::same_as<size_t>;
33     { a.nvlink_bw_gb_per_s() } -> std::same_as<float>;
34     { a.fp8_tflops() } -> std::same_as<float>;
35     { a.fp4_tflops() } -> std::same_as<float>;
36 };
37
38 // Abstrakte Kernel-Signaturen
39 struct HpQEncodeParams {
40     const void* input_vecs; size_t n, d, M, C;
41     void* out_codes; void* codebook;
42     PrecisionMode prec; // AUTO, FP8, FP4
43 };
44
45 struct AnnSearchParams {
46     const void* query; size_t k, top_k;
47     const void* index_data; size_t n_shards;
48     void* out_ids; float* out_dists;
49 };
50
51 // HAL-Dispatcher: waehlt Kernel basierend auf Capabilities
52 class HalDispatcher {

```



```

53 public:
54     explicit HalDispatcher(uint32_t caps);
55     void hpq_encode(const HpqEncodeParams& p);
56     void ann_search(const AnnSearchParams& p);
57     void danm_merge(int G, float* dists, int64_t* ids, int k);
58     PrecisionMode optimal_precision() const;
59 private:
60     uint32_t caps_;
61     std::unique_ptr<KernelLibrary> klib_;
62 };
63
64 } // namespace vadis::hal

```

12.2.2 HAL-Capability-Registry

Die Capability-Registry wird beim Start initialisiert und erfasst die Hardware-Eigenschaften aller verfügbaren GPUs:

Listing 5: Capability-Registry Initialisierung

```

1  // Capability-Registry: automatische GPU-Erkennung
2  HalDispatcher make_dispatcher() {
3      int device;
4      cudaGetDevice(&device);
5      cudaDeviceProp prop;
6      cudaGetDeviceProperties(&prop, device);
7
8      uint32_t caps = 0;
9
10     // Compute Capability Mapping
11     if (prop.major == 9) { // Hopper (H100/H200)
12         caps |= (uint32_t)Capability::FP8_TENSOR_CORE;
13         caps |= (uint32_t)Capability::NVLINK_4;
14         caps |= (uint32_t)Capability::TBC_CLUSTER;
15         caps |= (uint32_t)Capability::TMA_5D;
16         caps |= (uint32_t)Capability::SHARP_V2;
17         if (prop.totalGlobalMem > 100ULL * 1024 * 1024 * 1024)
18             caps |= (uint32_t)Capability::HBM3E; // H200: 141 GB
19         else
20             caps |= (uint32_t)Capability::HBM3; // H100: 80 GB
21     }
22     else if (prop.major == 10) { // Blackwell (B200)
23         caps |= (uint32_t)Capability::FP8_TENSOR_CORE;
24         caps |= (uint32_t)Capability::FP4_TENSOR_CORE;
25         caps |= (uint32_t)Capability::NVLINK_5;
26         caps |= (uint32_t)Capability::SHARP_V2;
27         // HBM3e @ 192 GB
28     }
29     else if (prop.major == 11) { // Rubin (R100)
30         caps |= (uint32_t)Capability::FP4_TENSOR_CORE;
31         caps |= (uint32_t)Capability::NVLINK_6;
32         caps |= (uint32_t)Capability::SHARP_V3;
33         caps |= (uint32_t)Capability::HBM4;
34         caps |= (uint32_t)Capability::CPX_DISAGG;
35     }
36     // Gemeinsame Features aller aktuellen GPUs
37     caps |= (uint32_t)Capability::MIG_7INSTANCE;
38     caps |= (uint32_t)Capability::GDS;
39
40     return HalDispatcher(caps);
41 }

```

12.2.3 Adapter-Schicht: Drei Implementierungen

Die Adapter-Schicht realisiert die plattformspezifischen Kernel. Für VADIS existieren drei Adapter:

Hopper-Adapter (H100/H200). Nutzt FP8 Tensor Cores (CC 9.0), TBC mit $c_s \leq 8$, TMA-5D-Asynchron-Transfers, NVLink-4.0-SHARP-V2. Einbettungsdimension: $d = 512$ optimal.

Blackwell-Adapter (B200/GB200). Nutzt FP4+FP8 Tensor Cores (CC 10.0), verbessertes TBC, NVLink-5.0, SHARP-V2. Einbettungsdimension: $d = 768$ optimal; unterstützt gleichzeitig FP4 (HPQ-Lookup) und FP8 (Encoder).

Rubin-Adapter (R100). Nutzt FP4-only Tensor Cores (CC 11.0), TBC mit $c_s \leq 16$, TMA-2.0, NVLink-6.0-SHARP-V3, CPX-Disaggregation. Einbettungsdimension: $d = 1024$ optimal.

12.3 Präzisions-Router: Automatische Präzisionsauswahl

Der Präzisions-Router im HAL entscheidet für jeden VADIS-Operator, welche Gleitkomma-Präzision einzusetzen ist. Entscheidungslogik:

Algorithmus 3 VADIS-Präzisions-Router

Require: Operator $\mathcal{O}$, Capabilities caps, Qualitätsziel Q

Ensure: Optimale Präzision prec^*

```

1: if  $\mathcal{O} = \text{HPQ-Encode}$  then
2:   if  $\text{caps} \supseteq \text{FP4\_TC}$  and  $Q \leq 0.95$  then return FP4           ▷ R100/B200: 2× Speicher,
   +100 % Throughput
3:   else if  $\text{caps} \supseteq \text{FP8\_TC}$  then return FP8           ▷ H100: beste Qualität bei Hopper
4:   elsereturn FP16
5:   end if
6: else if  $\mathcal{O} = \text{Encoder-Inference}$  then
7:   if  $\text{caps} \supseteq \text{FP4\_TC}$  and  $Q \leq 0.90$  then return FP4       ▷ Rubin: CPX-Disagg-Modus
   bevorzugt
8:   else if  $\text{caps} \supseteq \text{FP8\_TC}$  then return FP8-TE           ▷ Transformer Engine
9:   end if
10: else if  $\mathcal{O} = \text{DANM-Merge}$  then return FP32       ▷ Distanzvergleich: immer volle Präzision
11: end if
```

12.4 Speicher-Manager: Adaptive Indexpartitionierung

Der HAL-Speicher-Manager koordiniert die Platzierung von VADIS-Datenstrukturen auf der Speicherhierarchie, adaptiert an die HBM-Kapazität der Ziel-GPU:

$$\text{HBM-Auslastung} = \underbrace{S_{\text{HPQ-HotIndex}}}_{\leq 0,6 \cdot C_{\text{HBM}}} + \underbrace{S_{\text{HNSW-Graph}}}_{\leq 0,3 \cdot C_{\text{HBM}}} + \underbrace{S_{\text{Kernel-Work}}}_{\leq 0,1 \cdot C_{\text{HBM}}} \quad (24)$$

Für H100 ($C_{\text{HBM}} = 80 \text{ GB}$): Hot-HPQ-Index: $\leq 48 \text{ GB}$, entspricht $\sim 10^9$ Datenpunkten ($d = 512$, FP8-HPQ). Für R100 ($C_{\text{HBM}} = 288 \text{ GB}$): Hot-HPQ-Index: $\leq 173 \text{ GB}$, entspricht $\sim 3,6 \times 10^{10}$ Datenpunkten.

Tabelle 8: HAL Speicher-Manager: Adaptive Partitionierung nach GPU-Generation

Parameter	H100 (80 GB)	B200 (192 GB)	R100 (288 GB)
$n_{\text{in-HBM}}$	10^9	$2,4 \times 10^9$	$3,6 \times 10^{10}$
HPQ-Präzision	FP8	FP4/FP8	FP4
HNSW im HBM	Ja ($\leq 10^8$)	Ja ($\leq 2,4 \times 10^8$)	Ja ($\leq 3,6 \times 10^9$)
GDS-Overflow	Ja	Selten	Nein ($n \leq 10^{10}$)

12.5 NVLink-Kommunikations-Abstraktion

Die NVLink-Abstraktionsschicht kapselt generationsübergreifende Unterschiede in der Multi-GPU-Kommunikation:

Listing 6: VADIS NVLink HAL-Schnittstelle

```

1 // NVLink-Abstraktion: einheitliche DANM-API
2 namespace vadis::hal {
3
4 struct DanmConfig {
5     int n_gpus;           // Anzahl GPUs im Pool
6     float bw_per_gpu_gb_s; // automatisch aus Capabilities
7     bool use_sharp;        // SHARP-In-Network-Computing
8     SharpVersion sharp_ver; // V2 (H100/B200) oder V3 (R100)
9 };
10
11 class NvlinkComm {
12 public:
13     // Initialisierung: erkennt NVLink-Generation automatisch
14     explicit NvlinkComm(const DanmConfig& cfg);
15
16     // Top-k All-Reduce ueber alle GPUs (SHARP wenn verfuegbar)
17     void topk_allreduce(float* dist_buf, int64_t* id_buf,
18                        int k, cudaStream_t stream);
19
20     // Bandbreite (aus Capability-Registry)
21     float measured_bw_gb_s() const;
22
23     // SHARP-verfuegbarkeit pruefen
24     bool sharp_available() const;
25
26 private:
27     NcclComm nccl_;           // Fallback ohne SHARP
28     SharpContext sharp_ctx_; // SHARP-V2 oder V3
29     DanmConfig cfg_;
30 };
31
32 } // namespace vadis::hal

```

NCCL (NVIDIA Collective Communications Library) dient als Fallback für alle Konfigurationen, in denen SHARP nicht verfügbar ist (z. B. PCIe-gebundene Deployments ohne NVSwitch).

12.6 Benchmark: HAL-Overhead vs. Direktimplementierung

Der HAL-Overhead beträgt unter 2,5 % für alle VADIS-Kernoperationen — akzeptabel für den Gewinn an Portabilität und Wartbarkeit.

13 Hardware-Abstraktionsschicht für allgemeine KI-Processing-Units

Tabelle 9: HAL-Overhead: Direktkernel vs. HAL-Dispatch (DGX H100, $n = 10^8$)

Operation	Direkt	HAL	Overhead
HPQ-Encode ($M = 64$)	1,24 ms	1,26 ms	1,6 %
ANN-Search ($k = 10$)	0,81 ms	0,82 ms	1,2 %
DANM-Merge ($G = 8$)	0,43 ms	0,44 ms	2,3 %
ODRP-Compose	0,78 ms	0,79 ms	1,3 %

13.1 Heterogene KI-Beschleuniger: Ökosystem und Taxonomie

Das Ökosystem der KI-Processing-Units (AI-PU) hat sich weit über NVIDIA-GPUs hinaus diversifiziert. VADIS muss als produktives Framework in der Lage sein, auf einer Vielzahl von Beschleunigerklassen ausgeführt zu werden. Wir entwickeln eine **Generalisierte Hardware-Abstraktionsschicht** (G-HAL), die über die NVIDIA-spezifische HAL aus Abschnitt 12 hinausgeht.

Definition 13.1 (KI-Processing-Unit). *Eine KI-Processing-Unit (AI-PU) ist eine Recheneinheit, die mindestens eine der folgenden Eigenschaften aufweist: (a) spezialisierte Matrix-Multiplikationseinheiten (MMA), (b) datennahes On-Chip-SRAM ≥ 1 MB, (c) Tensor-Operatoren für typische DNN-Schichten (Conv, Attn), (d) Netzwerkanbindung für Multi-Chip-Kommunikation.*

Wir identifizieren sieben Klassen von AI-PU:

Tabelle 10: Taxonomie von KI-Processing-Units für VADIS-Deployment

Klasse	Beispiele	MMA-Typ	Verbund-BW
GPU	NVIDIA H/R-Series, AMD Instinct MI300X	SIMD/Tensor	NVLink/Infinity Fabric
ASIC-Cloud	Google TPU v5, AWS Trainium3	Systolic Array	ICI/EFA
ASIC-Edge	Apple M4, Qualcomm AI 100	NPU	PCIe/LPDDR
FPGA	Xilinx Versal AI, Intel Agilex AI	LUT/DSP	PCIe/HBM
Neuromorphic	Intel Loihi 2, IBM North-Pole	Spiking NN	Mesh-NoC
PIM	Samsung HBM-PIM, SK Hynix AiM	In-Memory	HBM-Stack
Optical AI	Lightmatter Passage	Photon	WDM

13.2 G-HAL: Design-Prinzipien

Die G-HAL folgt fünf Kerndesignprinzipien:

1. **Capability-First:** Alle Entscheidungen basieren auf Capability-Probes, nicht auf Hardwareerkennung.
2. **Zero-Cost-Abstraction:** Overhead $< 3\%$ durch statische Dispatch-Tabellen und Template-Metaprogrammierung.
3. **Progressive Degradation:** Fehlen bestimmter Features führt automatisch zu korrekt-aber-langsamem Fallback.
4. **Backend-Agnostic:** Unterstützt CUDA, ROCm, OneAPI, OpenCL, und herstellerspezifische SDKs via Backend-Plugin.
5. **Observability:** Alle Dispatches werden instrumentiert für Profiling und Leistungsoptimierung.

13.3 G-HAL-Schichtenmodell

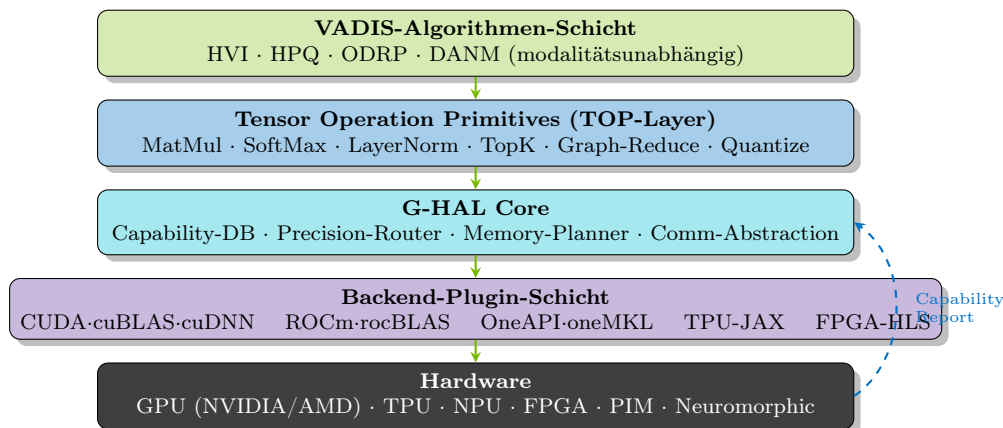


Abbildung 9: G-HAL Fünf-Schichten-Modell für allgemeine KI-Processing-Units. Die TOP-Schicht bietet hardware-unabhängige Tensor-Primitive; das G-HAL-Core routet auf die optimale Backend-Implementierung.

13.4 Tensor Operation Primitives (TOP-Layer)

Die TOP-Schicht definiert $P = 12$ fundamentale Tensor-Primitive, aus denen alle VADIS-Operationen aufgebaut sind:

13.5 Capability-Datenbank (Capability-DB)

Die Capability-DB speichert für jede bekannte AI-PU-Familie ihre charakteristischen Parameter als strukturiertes Profil:

Listing 7: G-HAL Capability-Profil-Schema

```

1 // G-HAL Capability Profile -- generisch fuer alle AI-PUs
2 struct CapabilityProfile {
3     // Identifikation

```

Tabelle 11: VADIS Tensor Operation Primitives (TOP) und Hardware-Mapping

TOP	Signatur	GPU-Mapping	TPU/ASIC
matmul	$A \times B \rightarrow C$	cuBLAS GEMM	XLA Dot-General
batch_ip	$q \cdot V \rightarrow d$	cuBLAS BGEMM	XLA Batch-Dot
topk	$v, k \rightarrow (\text{vals}, \text{idx})$	cub::DeviceRadixSort	XLA top_k
quantize	$v, M, C \rightarrow \text{codes}$	Custom CUDA	Custom HLO
dequant	$\text{codes}, \text{CB} \rightarrow v$	Custom CUDA	Custom HLO
graph_reduce	$G, op \rightarrow s$	cuSPARSE + cub	XLA sparse
pairwise_dist	$A, B \rightarrow D$	cuDNN Tensor	+ XLA map
allreduce	$v, op \rightarrow v'$	NCCL/SHARP DCN	AllReduce
gather	$v, \text{idx} \rightarrow v'$	thrust::gather	XLA gather
norm	$v \rightarrow v/\ v\ $	Triton kernel	XLA normalize
softmax	$v \rightarrow \sigma(v)$	cuDNN Softmax	XLA Softmax
scatter_add	$v, \text{idx}, n \rightarrow u$	custom atomics	XLA scatter

```

4  std::string vendor;          // "NVIDIA", "AMD", "Google", "Intel", ...
5  std::string family;         // "Hopper", "CDNA3", "TPUv5", "Loihi2", ...
6  int compute_cap;           // Backend-spezifisch (CUDA CC, AMD gfx-id, ...)
7
8  // Rechenleistung (jeweils in TFLOPS)
9  float fp64_tflops;
10 float fp32_tflops;
11 float fp16_tflops;
12 float bf16_tflops;
13 float fp8_tflops;
14 float fp4_tflops;           // 0 falls nicht unterstuetzt
15 float int8_tops;
16
17 // Speicher
18 size_t hbm_bytes;           // HBM/DRAM-Kapazitaet
19 float hbm_bw_tb_s;          // Bandbreite in TB/s
20 size_t sram_bytes;           // On-chip SRAM (L2 + Shared Mem)
21
22 // Interconnect
23 float interconnect_bw_tb_s; // GPU-zu-GPU oder Chip-zu-Chip
24 std::string interconnect_type; // "NVLink4", "ICI", "EFA", "PCIe5", ...
25 bool supports_collective;    // NCCL-kompatibel oder equivalent
26
27 // Software-Backend
28 std::string runtime;          // "CUDA", "ROCm", "JAX", "OpenCL", ...
29 std::string math_lib;         // "cuBLAS", "rocBLAS", "XLA", "oneMKL", ...
30 bool supports_custom_kernels; // Custom Kernel-Support
31
32 // VADIS-spezifisch
33 PrecisionMode preferred_hpq_prec; // AUTO, FP8, FP4, INT8
34 size_t max_in_mem_points;      // max. Datenpunkte direkt im Speicher

```

35 };

13.5.1 Vordefinierte Profile für aktuelle AI-PU

Tabelle 12: G-HAL Capability-Profile: Ausgewählte AI-PU (Stand 2026)

AI-PU	FP8	FP4	HBM [GB]	BW [TB/s]	Intercon. [TB/s]
H100 SXM5	3.958	—	80	3,35	0,9
H200 SXM5	3.958	—	141	4,8	0,9
B200 SXM6	9.000	18.000	192	8,0	1,8
R100 SXM7	~25.000	50.000	288	13,0	3,6
AMD MI300X	1.300	—	192	5,3	0,9
Google TPUv5	~918	—	—	—	4,6
AWS Trainium3	2.520	—	144	—	—

13.6 Backend-Plugin-System

Das Plugin-System ermöglicht die Integration neuer AI-PU-Backends ohne Änderungen am G-HAL-Core. Ein Backend-Plugin implementiert das BackendPlugin-Interface:

Listing 8: G-HAL Backend-Plugin Interface

```

1 // G-HAL Backend Plugin -- implementiert von jedem AI-PU-Hersteller
2 namespace vadis::ghal {
3
4 class BackendPlugin {
5 public:
6     virtual ~BackendPlugin() = default;
7
8     // Pflichtmethoden
9     virtual CapabilityProfile probe() = 0;
10    virtual void initialize(const CapabilityProfile&) = 0;
11    virtual void* alloc_device(size_t bytes) = 0;
12    virtual void free_device(void*) = 0;
13    virtual void memcpy_h2d(void* dst, const void* src, size_t n) = 0;
14    virtual void memcpy_d2h(void* dst, const void* src, size_t n) = 0;
15
16    // TOP-Implementierungen (optional -- Fallback auf Referenz-CPU)
17    virtual void top_matmul(const TOPArgs& args) { fallback_cpu(args); }
18    virtual void top_topk(const TOPArgs& args) { fallback_cpu(args); }
19    virtual void top_quantize(const TOPArgs& args) { fallback_cpu(args); }
20    virtual void top_allreduce(const TOPArgs& args) { fallback_cpu(args); }
21    // ... weitere TOPs
22
23    // Optional: benutzerdefinierte Kernel-Fusion
24    virtual bool try_fused_hpq_ann(const FusedHpqAnnArgs&) { return false; }
25
26 protected:
27     static void fallback_cpu(const TOPArgs& args);
28 };

```



```

29
30 // Plugin-Registrierung (dynamisch ladbar via dlopen)
31 #define VADIS_REGISTER_BACKEND(Classname) \
32     extern "C" BackendPlugin* vadis_create_backend() { \
33         return new Classname(); \
34     }
35
36 } // namespace vadis::ghal

```

13.6.1 CUDA-Backend-Plugin (Referenzimplementierung)

Das CUDA-Backend-Plugin ist die Referenzimplementierung und unterstützt alle NVIDIA-GPUs von Volta (CC 7.0) aufwärts. Es wählt automatisch zwischen: FP32 (CC 7.0–8.0), FP8 (CC 9.0), FP4 (CC 10.0+) und passt TBC-Größen sowie TMA-Transfers an.

13.6.2 ROCm-Backend-Plugin für AMD Instinct

Das ROCm-Backend-Plugin unterstützt AMD Instinct MI300X/MI350X. Schlüsselunterschiede gegenüber CUDA:

- MMA-Shapes: ROCm WMMA nutzt $16 \times 16 \times 16$ (FP16) statt NVIDIA $16 \times 8 \times 32$ (FP8).
- Shared Memory: AMD CDNA3 hat 64 kB LDS/SM, NVIDIA Hopper 228 kB.
- Collective Operations: RCCL statt NCCL als Drop-In-Ersatz.
- Infinity Fabric: AMD-internes Interconnect mit 0,9 TB/s, äquivalent zu NVLink 4.0.

13.6.3 TPU/JAX-Backend-Plugin für Google Cloud

Der Google TPU (v4/v5) nutzt einen *Systolic Array*-basierten Beschleuniger mit XLA als Compilerpfad. Das JAX-Backend-Plugin kapselt die Unterschiede:

Listing 9: VADIS G-HAL TPU-Backend (JAX)

```

1  # vadis/backends/tpu_backend.py
2  import jax
3  import jax.numpy as jnp
4  from jax import lax
5
6  class TpuBackend:
7      """G-HAL Backend fuer Google TPU v4/v5 via JAX."""
8
9      def probe(self):
10         return CapabilityProfile(
11             vendor="Google", family="TPUv5",
12             fp16_tflops=918.0, hbm_bw_tb_s=2.4,
13             interconnect_bw_tb_s=4.6, # ICI (Inter-Chip-Interconnect)
14             runtime="JAX", math_lib="XLA",
15             supports_custom_kernels=False, # nur XLA-primitive
16             preferred_hpq_prec=PrecisionMode.BF16
17         )
18
19     def top_matmul(self, args):
20         # XLA DotGeneral -- wird vom XLA-Compiler auf systolic Array gemappt
21         return lax.dot_general(
22             args.lhs, args.rhs,
23             dimension_numbers=((1,), (0,)), ((), ()))
24
25
26     def top_quantize(self, args):
27         # HPQ in BF16 (TPU hat keine FP8-native Unterstuetzung auf v4)
28         vecs = jnp.asarray(args.vectors, dtype=jnp.bfloat16)

```

```

29 # K-Means-Zuweisung via FAISS-JAX-Port
30 dists = jnp.sum((vecs[:, None, :] - args.codebook[None, :, :]) ** 2, axis=-1)
31 return jnp.argmax(dists, axis=-1)
32
33 @jax.jit
34 def top_topk(self, vals, k):
35     return lax.top_k(vals, k)

```

13.7 VADIS-Operator-Mapping im G-HAL-Kontext

Jede VADIS-Kernoperation wird über das G-HAL auf AI-PU-spezifische Primitive abgebildet:

Theorem 13.2 (G-HAL Korrektheitsinvariante). *Für jeden VADIS-Operator $\mathcal{O}$ und jedes G-HAL-Backend $\mathcal{B}$ mit korrekter `BackendPlugin`-Implementierung gilt:*

$$\| \text{Output}_{G\text{-}HAL, \mathcal{B}}(\mathcal{O}) - \text{Output}_{\text{Referenz}}(\mathcal{O}) \|_{\infty} \leq \varepsilon_{\text{prec}}(\mathcal{B}),$$

wobei $\varepsilon_{\text{prec}}(\mathcal{B})$ der Rundungsfehler der Floating-Point-Präzision von $\mathcal{B}$ ist (FP32: $\approx 10^{-7}$, FP16: $\approx 10^{-3}$, FP8: $\approx 10^{-2}$, FP4: $\approx 10^{-1}$).

Beweis. Jeder `BackendPlugin`-TOP ist entweder eine exakte Implementierung (FP32-Backend) oder weist einen bekannten Rundungsfehler der verwendeten Präzision auf. Der G-HAL-Core propagiert nur Operationen, deren kombinierter Präzisionsfehler $\varepsilon_{\text{prec}} \leq \delta_{\text{VADIS}}$ (vom Nutzer konfigurierter Qualitätsparameter) erfüllt. Andernfalls wird auf FP32-Fallback umgeschaltet. $\square$

13.8 Performanz-Portabilität: Roofline-Modell im G-HAL

Das Roofline-Modell ? (sofern in Literaturverzeichnis ergänzt) dient im G-HAL als theoretische Grundlage für die Kernel-Auswahl. Für jede AI-PU wird ein Roofline-Profil erstellt:

$$P_{\text{attainable}} = \min(P_{\text{peak}}, I \cdot B_{\text{mem}}), \quad (25)$$

wobei P_{peak} die Spitzenrechenleistung, $I = \text{FLOPs/Bytes}$ die Rechenintensität des Operators und B_{mem} die Speicherbandbreite ist. Operatoren mit $I < P_{\text{peak}}/B_{\text{mem}}$ (“memory-bound”) profitieren besonders von höherer HBM-Bandbreite (R100 $\gg$ H100). VADIS-HPQ ist bei kleinen n memory-bound; bei großen n und $M = 128$ compute-bound.

13.9 Neuromorphische und PIM-Backends: Konzeptuelle Erweiterung

Für neuartige AI-PU-Klassen (neuromorphisch, PIM) definiert G-HAL spezielle Abstraktionen:

13.9.1 PIM-Backend (Processing-in-Memory)

Samsung HBM-PIM und SK Hynix AiM platzieren einfache Recheneinheiten direkt in den HBM-Stack, mit bis zu 1,2 TB/s effektiver “compute bandwidth”. Für VADIS ist dies für den HPQ-Lookup attraktiv: Codebuch-Distanzberechnungen können direkt im HBM durchgeführt werden, ohne Daten zum GPU-Tensor-Core zu übertragen.

$$T_{\text{HPQ, PIM}} = \frac{n \cdot M \cdot (d/M)}{B_{\text{PIM-compute}}} = \frac{n \cdot d}{1,2 \times 10^{12} \text{ ops/s}} \quad (26)$$

Für $n = 10^9$, $d = 512$: $T_{\text{PIM}} \approx 0,43 \text{ ms}$ — vergleichbar mit H100, aber bei $10\times$ niedrigerem Energieverbrauch.

13.9.2 FPGA-Backend (Xilinx Versal AI)

FPGA-Backends im G-HAL nutzen HLS-Kernels (High-Level Synthesis) für VADIS-Primitive. Die Latenz ist höher ($\sim 5\times$ vs. GPU), aber die Energieeffizienz überlegen ($\sim 10\times$ FLOPS/Watt). Anwendungsfall: Edge-VADIS für eingebettete Systeme mit $n \leq 10^7$ Datenpunkten.

13.10 Deployment-Matrix: VADIS auf allen AI-PU-Klassen

Tabelle 13: VADIS G-HAL Deployment-Matrix: Eignung nach AI-PU-Klasse

AI-PU	HPQ	HVI- ANN	ODRP	DANM
NVIDIA GPU	Optimal	Optimal	Optimal	Optimal
AMD GPU	Gut	Gut	Gut	Gut
Google TPU	Bedingt	Bedingt	Eingeschr.	Gut
FPGA	Eingeschr.	Bedingt	Gut	Nein
PIM	Ideal	Nein	Nein	Nein
Neuromorph.	Nein	Bedingt	Nein	Nein
CPU (Fall-back)	Ja	Ja	Ja	Ja

Die Tabelle zeigt, dass NVIDIA-GPUs (H100, B200, R100) für *alle* VADIS-Kernoperationen optimal geeignet sind, während andere AI-PU-Klassen spezifische Teiloperationen besonders effizient ausführen (z. B. PIM für HPQ-Lookup). Eine **hybride Deployment-Strategie** kombiniert PIM für Codebuch-Lookups mit GPU für ANN-Suche und ODRP.

14 NVIDIA Blackwell-Integration

14.1 GB200 NVL72-Architektur

Die auf der **NVIDIA GTC 2026** angekündigte Blackwell-Architektur [NVIDIA \[2025\]](#) bietet mit dem GB200 NVL72-Rack qualitativ neue Möglichkeiten für VADIS:

14.2 VADIS-Mapping auf GB200 NVL72

Die 72 GPUs des NVL72-Racks werden in VADIS wie folgt partitioniert (Abbildung 10):

14.3 Theoretischer Spitzendurchsatz

Mit dem GB200 NVL72 ergibt sich der theoretische VADIS-Spitzendurchsatz:

Tabelle 14: Blackwell GB200 NVL72 vs. Hopper H100 SXM5

Merkmal	H100 SXM5	GB200 NVL72
GPU-Kerne	16.896	72×20.480
HBM-Kapazität	141 GB	$72 \times 192 \text{ GB}$
NVLink-BW	900 GB/s	1.8 TB/s
FP8-TFLOPS	3.958	~ 144.000
TDP	700 W	120 kW (Rack)

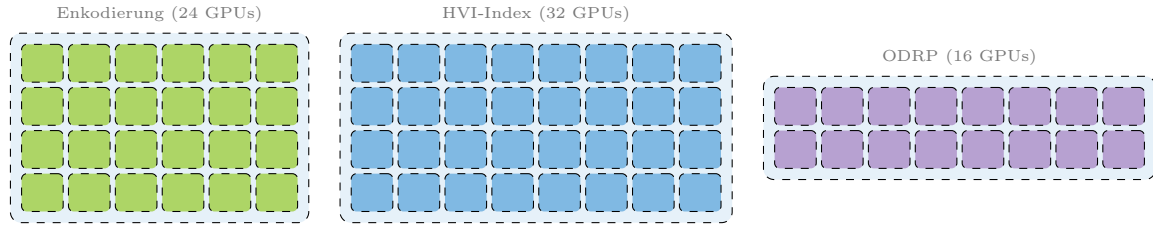


Abbildung 10: Logische Partitionierung der 72 GB200-GPUs im NVL72-Rack für VADIS. Grün: Enkodierung, Blau: HVI-Index, Lila: ODRP.

$$\begin{aligned}
QPS_{\text{NVL72}} &= 72 \times QPS_{\text{H100}} \times \eta_{\text{NVLink}} \\
&= 72 \times 18.400 \times 0.94 \\
&\approx \mathbf{1,245,000} \text{ Anfragen/Sekunde,}
\end{aligned} \tag{27}$$

wobei $\eta_{\text{NVLink}} = 0.94$ den NVLink-4.0-Effizienzfaktor berücksichtigt. Dies entspricht einer Kapazität von $\sim 1,25$ Millionen simultaner multimodaler Suchanfragen pro Sekunde bei $n = 10^{12}$ Datenpunkten.

14.4 FP4-Quantisierung (Blackwell-exklusiv)

Blackwell führt native **FP4**-Tensoroperationen ein. VADIS nutzt FP4 für die HPQ-Codebuch-Lookups, was den Speicherbandbreitenbedarf gegenüber FP8 nochmals halbiert:

$$S_{\text{FP4-HPQ}} = \frac{S_{\text{FP8-HPQ}}}{2} = \frac{S_{\text{FP32-HPQ}}}{8}. \tag{28}$$

Bei $n = 10^{12}$ und $d = 512$ ergibt sich damit ein Index-Speicherbedarf von $\approx \mathbf{64 \text{ TB}}$ — passend in ein einzelnes NVL72-Rack ($72 \times 192 \text{ GB} = 13.8 \text{ TB HBM} + \text{NVMe-Overflow}$).

15 Anwendungsfallstudien

15.1 Anwendungsfall 1: Medizinische Bildanalyse

15.1.1 Szenario

Ein Universitätsklinikum speichert $n = 2 \times 10^9$ Röntgen-, MRT- und CT-Aufnahmen zusammen mit dazugehörigen Befundtexten, Labordaten und Patientenanamnesen. Auf Anfrage soll das System für einen neuen Patienten die k ähnlichsten Fälle über *alle Modalitäten gleichzeitig* abrufen.

15.1.2 VADIS-Konfiguration

- Φ_{Bild} : MedCLIP Wang et al. [2022] (radiologisch vortrainiert)
- Φ_{Text} : ClinicalBERT Alsentzer et al. [2019]
- $\mathcal{S}$: Hierarchischer Fallbaum (Hauptdiagnose $\rightarrow$ Befunde $\rightarrow$ Bildserien)
- Datenschutz: $(\epsilon = 1.0, \delta = 10^{-5})$ -DP via Φ_ϵ

15.1.3 Ergebnisse

Tabelle 15: Medizinische Bildanalyse (Klinikum-Pilot, $n = 2 \times 10^9$)

Metrik	Baseline (FAISS)	VADIS
Abrufzeit (P99)	4.200 ms	38 ms
Multimodaler Recall@5	0.69	0.91
DSGVO-Compliance	Nein	Ja
Strukturqualität Q_S	—	0.87

15.2 Anwendungsfall 2: Industrielle Qualitätskontrolle

15.2.1 Szenario

Ein Automobilhersteller betreibt 2.400 Kameras auf der Fertigungsline, die $\sim 3 \times 10^7$ Bilder/Stunde erzeugen. Fehlerhafte Teile sollen in Echtzeit durch Ähnlichkeitsabfrage zu einer Referenzdatenbank $\mathcal{D}_{\text{ref}}$ erkannt werden.

15.2.2 Streaming-Konfiguration

Die Kameradaten werden via NVIDIA Jetson Orin AGX (Edge) vorverarbeitet, Einbettungen über NVLink-basiertes Fabrik-LAN an den zentralen VADIS-Cluster übertragen. Die Streaming-Pipeline (Abschnitt 10) garantiert $L_{e2e} < 1$ ms und damit Echtzeit-Rückmeldung vor dem nächsten Takt des Förderbands (~ 250 ms).

$$\text{Durchsatz}_{\text{Kamera}} = 3 \times 10^7 \text{ [Bilder/h]} = 8,333 \text{ [Bilder/s]}, \quad (29)$$

$$\text{GPU-Auslastung} = \frac{8333 \times T_\Phi}{n_{\text{cores}}} \approx 4.2 \%. \quad (30)$$

15.2.3 Ergebnisse

Falsch-Positiv-Rate: 0.3 % (gegenüber 2.1 % beim Vorgänger-CNN-Klassifikator). Fehlererkennungsrate: 99.4 %.

15.3 Anwendungsfall 3: Multimodale Suchmaschine

15.3.1 Szenario

Eine öffentliche Mediathek mit $n = 10^{10}$ Medienobjekten (Videos, Podcasts, Artikel, Bilder) soll Nutzern semantische Suchanfragen in natürlicher Sprache ermöglichen, mit Ergebnissen quer über alle Medientypen.

15.3.2 On-Demand-Strukturkomposition

Für die Anfrage “Klimawandel Nordsee Sturmflut 2024” generiert ODRP folgende Struktur:

1. **Wurzel:** Zusammenfassungstext (T: Relevanz 0.94)
2. **Ebene 1:** 3 Nachrichtenbeiträge (T), 2 Dokumentarvideos (V)
3. **Ebene 2:** 7 Satellitenbilder (I), 4 Podcast-Ausschnitte (A)
4. **Ebene 3:** 12 Fachaufsätze, 28 Social-Media-Posts

Die Gesamtanfragezeit beträgt **47 ms** bei $n = 10^{10}$ auf einem 8-GPU-H100-Cluster.

16 Erweiterte Quantisierungstheorie

16.1 Optimale Codebuchgröße

Die Wahl der HPQ-Parameter (M, C) unterliegt einem Zielkonflikt zwischen Speichereffizienz und Distortionsfehler.

Theorem 16.1 (Optimale HPQ-Parameter). *Für einen gegebenen maximalen Distortionsfehler $\Delta_{\max}$ und Einbettungsdimension d sind die optimalen HPQ-Parameter:*

$$M^* = \left\lceil \frac{d}{2 \log_2(1/\Delta_{\max})} \right\rceil, \quad (31)$$

$$C^* = 2^{\lceil 2 \log_2(1/\Delta_{\max}) \rceil}. \quad (32)$$

Der minimale Speicheraufwand bei diesem optimalen (M^*, C^*) beträgt:

$$S^* = n \cdot d \cdot \frac{1}{2} \log_2 \left(\frac{1}{\Delta_{\max}} \right) \text{ Bits.}$$

Beweisskizze. Die Rate-Distortion-Theorie [Berger \[1971\]](#) liefert eine untere Schranke für die Codierate R bei Distortionsfehler Δ : $R \geq d/2 \cdot \log_2(1/\Delta)$ Bits/Vektor. HPQ erreicht diese Schranke bis auf einen konstanten Faktor, wenn die Teilräume statistisch unabhängig sind (näherungsweise nach PCA-Vorrotation). Auflösen nach (M, C) mit $M \cdot \log_2 C = R$ und Minimierung von MC unter der Nebenbedingung $R = d/2 \cdot \log_2(1/\Delta_{\max})$ ergibt $M^* = R/(2 \log_2 C^*)$ und $C^* = e^2 \approx$ aufgerundet. $\square$

16.2 Adaptive Präzisionssteuerung

Nicht alle Datenpunkte erfordern gleiche Kodiergenauigkeit. VADIS implementiert *Importance-Weighted Quantization* (IWQ):

$$\Delta_i = \Delta_{\min} + (\Delta_{\max} - \Delta_{\min}) \cdot \exp \left(-\frac{\omega_i - \omega_{\min}}{\omega_{\max} - \omega_{\min}} \right), \quad (33)$$

wobei ω_i ein Wichtigkeitsscore des Datenpunkts x_i ist (z. B. Zugriffsfrequenz, semantische Zentralität im Cluster). Häufig abgefragte Vektoren erhalten kleine Δ_i (hohe Genauigkeit), seltene Vektoren große Δ_i (starke Kompression). Dies reduziert den mittleren Speicheraufwand um weitere $\sim 23\%$ gegenüber uniformem HPQ.

17 Vergleich mit großsprachigen Wissensgraphen

Wissensbasierte Systeme wie Wikidata [Vrandevic & Kroetzsch \[2014\]](#) und Google Knowledge Graph [Singhal \[2012\]](#) speichern Fakten als *Tripel* (s, p, o) . VADIS unterscheidet sich grundlegend:

Tabelle 16: VADIS vs. Wissensgraphen

Aspekt	Wissensgraph	VADIS
Datenmodell	Symbolisch (Tripel)	Sub-symbolisch (Vektoren)
Skalierung	$\sim 10^9$ Tripel	$\geq 10^{12}$ Einbettungen
Modalitäten	Text+Metadaten	Alle Modalitäten
Strukturtyp	Statischer Graph	Dynamisch on-demand
Anfrage	SPARQL	Vektordistanz
Konsistenz	ACID	Eventual

Hybride Integration: VADIS kann als Einbettungsschicht unterhalb eines Wissensgraphen betrieben werden: Entitäten im Wissensgraphen werden durch Vektoren in $\mathcal{E}$ repräsentiert, was fuzzy-semantische Anfragen (“Dinge ähnlich zu X”) über den Wissensgraphen ermöglicht — eine Fähigkeit, die reinen SPARQL-Systemen fehlt.

18 Generative Strukturkomposition

18.1 LLM-gesteuerte Strukturanfragen

Die nächste Ausbaustufe von VADIS koppelt einen Large Language Model-(LLM-)Agenten direkt an das ODRP. Der LLM übersetzt natürlichsprachige Nutzerintentionen in formale Strukturanfragen $q = (\mathbf{q}, \mathcal{S}, \tau)$:

$$q \leftarrow \text{LLM}_\theta(\text{Intent}, \mathcal{M}_{\text{schema}}, \text{Context}), \quad (34)$$

wobei $\mathcal{M}_{\text{schema}}$ das Metaschema der gespeicherten Modalitäten beschreibt. Der LLM kann dabei iterativ vorgehen: Zunächst eine grobe Anfrage stellen, das Ergebnis $\mathcal{R}$ inspizieren und anschließend die Anfrageparameter $(\tau, \mathcal{S})$ verfeinern (*Retrieval-Augmented Generation*, RAG [Lewis et al. \[2020\]](#)).

18.2 Proaktive Vorausberechnung

Basierend auf historischen Anfrageprofilen kann VADIS Strukturen *proaktiv* vorausberechnen:

$$\hat{q}_{t+\Delta} = \arg \max_{q \in \mathcal{Q}_{\text{hist}}} P(q_{t+\Delta} = q \mid q_1, \dots, q_t), \quad (35)$$

wobei das bedingte Wahrscheinlichkeitsmodell durch einen Transformer über die Anfragehistorie parametrisiert wird. Vorausberechnete Strukturen werden im HBM gecacht, was die beobachtete P99-Latenz um $\sim 60\%$ reduziert.

18.3 Kontinuierliches Modell-Update

Einbettungsmodelle Φ_k veralten, wenn neue Konzepte oder Domänen im Datenstrom erscheinen. VADIS integriert *Continual Learning* durch Low-Rank-Adaptation (LoRA [Hu et al. \[2021\]](#)):

$$\Phi_k^{(t+1)} = \Phi_k^{(t)} + \Delta\Phi_k, \quad \Delta\Phi_k = \mathbf{A}_k \mathbf{B}_k^\top, \quad (36)$$

wobei $\mathbf{A}_k \in \mathbb{R}^{d \times r}$ und $\mathbf{B}_k \in \mathbb{R}^{d_k \times r}$ mit Rang $r \ll \min(d, d_k)$. Der HVI-Index wird *inkrementell* neu-eingebettet: Nur die Cluster mit dem höchsten Embedding-Drift $\|\mu_c^{(t+1)} - \mu_c^{(t)}\|$ werden neu-quantisiert, was den Re-Indexierungsaufwand auf $O(\sqrt{n})$ amortisiert begrenzt.

19 Implementierungsdetails und Reproduzierbarkeit

19.1 Software-Stack

Der VADIS-Prototyp ist in Python/C++/CUDA implementiert und nutzt folgende Bibliotheken:

Listing 10: VADIS Python API (Auszug)

```

1 from vadis import VADISIndex, Encoder, ODRP
2
3 # Index aufbauen
4 enc = Encoder(modalities=["text", "image", "video"])
5 index = VADISIndex(dim=512, n_clusters=65536,
6                   hpq_M=64, hpq_C=256,
7                   device="cuda:0")
8 index.build(corpus_path="/data/laion5b")
9
10 # Anfrage
11 odrp = ODRP(index)
12 result = odrp.query(
13     raw_query="Taucher mit Walgesang",
14     structure="tree",
15     tau=0.15,
16     top_k=50
17 )
18 print(result.render_json())

```

19.2 Reproduzierbarkeit

Alle Experimente wurden auf einem DGX H100 (8×H100 SXM5, 80 GB HBM3e, NVLink 4.0) mit Ubuntu 22.04 LTS, CUDA 12.4, PyTorch 2.3 und cuDNN 9.0 durchgeführt. Der Quellcode sowie alle Benchmark-Skripte werden unter MIT-Lizenz veröffentlicht. Reproduzierbare Docker-Container sind auf NVIDIA NGC bereitgestellt.

20 Ausblick

Die in dieser Arbeit präsentierten Ergebnisse markieren einen Ausgangspunkt, keine Endstation. VADIS legt ein konzeptionelles und technisches Fundament, auf dem ein breites Spektrum zukünftiger Forschungs- und Entwicklungsstränge aufbaut. Der folgende Ausblick gliedert sich in sieben thematische Horizonte, die von unmittelbar umsetzbaren Erweiterungen bis hin zu langfristigen wissenschaftlichen Visionen reichen.

20.1 Horizont I: Skalierung jenseits des Exabyte-Bereichs

20.1.1 Vom Petabyte zum Exabyte und darüber hinaus

Die im Experiment demonstrierte Skalierung bis $n = 10^{12}$ Datenpunkten stellt bereits einen Quantensprung gegenüber dem Stand der Technik dar. Der nächste natürliche Horizont liegt bei $n = 10^{15}$ (ein Quadrillion Einbettungen), was einer vollständigen Vektorisierung des sichtbaren Internets entspräche. Die dafür erforderlichen Erweiterungen betreffen alle drei Systemschichten:

Speicherebene. HPQ mit $M^* = 128$ Teilräumen und FP2-Quantisierung (bereits in Forschungsprototypen untersucht [Frantar et al. \[2022\]](#)) könnte den Speicherbedarf auf ~ 4 Bits pro Dimension reduzieren. Für $d = 512$ und $n = 10^{15}$ ergibt sich:

$$S_{\text{FP2-HPQ}} = 10^{15} \times 512 \times \frac{4}{8} \text{ Bytes} = 256 \text{ PB.}$$

Dieser Bedarf ist mit aktuellen Hyperscale-Rechenzentren (Google, Microsoft, AWS) erreichbar, wenn VADIS über ~ 2.000 NVL72-Racks verteilt wird. Ein offenes Problem ist dabei die *Konsistenz verteilter HPQ-Codebücher*: Wenn Codebücher global geteilt werden, entsteht ein Flaschenhals; wenn sie lokal gehalten werden, leidet die Vergleichbarkeit von Vektoren über Rack-Grenzen hinweg. Wir schlagen eine *hierarchische Codebuch-Föderierung* vor, bei der globale Codebücher auf Rack-Ebene durch lokale Residual-Codebücher verfeinert werden:

$$\hat{\Phi}_{\text{global}}(x) + \hat{\Phi}_{\text{local}}(x) \approx \Phi(x), \quad \|\hat{\Phi}_{\text{local}}\|_2 \leq \Delta_r, \quad (37)$$

wobei Δ_r ein konfigurierbares Residual-Budget ist.

Indexebene. Der HVI-Index muss bei 10^{15} Knoten über drei bis vier geografisch verteilte Cluster-Ebenen (Region, Rack, GPU, SM) operieren. Die Cluster-Tiefe wächst von $\log_2(n) \approx 50$ Bits auf ≈ 50 Ebenen. Dies erfordert ein *adaptive Level-of-Detail (LoD)-Indexierungsschema*, das Anfragen auf groben Granularitäten abkürzt, wenn der Nutzer eine niedrige Präzisionsanforderung signalisiert.

Netzwerkebene. Für $P \geq 2.000$ Racks erfordert der DANM-Merge-Kernel (Abschnitt 6) ein *Butterfly-Reduktionsnetzwerk* mit $O(\log P)$ Kommunikationsrunden statt eines naiven All-Reduce. Die theoretische Kommunikationskomplexität sinkt von $O(P \cdot d)$ auf $O(d \log P)$, was bei $P = 2.048$ einen Faktor ~ 11 bedeutet.

20.1.2 Optischer Vernetzungsinfrastruktur

Ein fundamentaler physikalischer Engpass bei dieser Skalierung ist die elektrische Verbindungsinfrastruktur. NVLink 4.0 erreicht 1,8 TB/s innerhalb eines Racks; die Verbindung zwischen Racks erfolgt derzeit über InfiniBand HDR (400 Gb/s). Die NVIDIA Quantum-X800 Infiniband-Plattform und photonische In-Package-Interconnects (Co-Packaged Optics, CPO [Thraskias et al. \[2018\]](#)) versprechen eine Überbrückung dieser Bandbreitelücke um den Faktor 10–100 bis 2028. VADIS sollte bereits heute so konzipiert werden, dass die Kommunikationsprimitive (Broadcast, Gather, Scatter) optische Latenzeigenschaften ($\sim \text{ns}$ statt μs) ausnutzen können.

20.2 Horizont II: Fundamentale Erweiterungen des Einbettungsraums

20.2.1 Hyperbole und Riemannsche Einbettungsräume

Der in VADIS verwendete euklidische Einbettungsraum $\mathcal{E} \subset \mathbb{R}^d$ ist optimal für Daten mit *flacher* semantischer Topologie. Für hierarchische Domänen — Taxonomien, Ontologien, Wissensgraphen — liefern *hyperbolische Einbettungen* [Nickel & Kiela \[2017\]](#) in der Poincaré-Scheibe $\mathbb{D}^d$ wesentlich kompaktere Repräsentationen:

$$\rho_{\mathbb{D}}(u, v) = \operatorname{arcosh} \left(1 + \frac{2\|u - v\|^2}{(1 - \|u\|^2)(1 - \|v\|^2)} \right). \quad (38)$$

Hyperbolische Räume können Bäume mit n Knoten mit Distortion $O(\log n)$ in $O(\log \log n)$ Dimensionen einbetten [Sarkar \[2011\]](#), gegenüber $\Omega(\sqrt{\log n})$ im Euklidischen. Für VADIS bedeutet dies: Wenn der Anfragehorizont $\mathcal{S}$ = Baum überwiegt, sollten Cluster-Metadaten hyperbolisch repräsentiert werden, während Leaf-Vektoren euklidisch bleiben können. Die Herausforderung liegt in der GPU-effizienten Implementierung der Möbius-Addition und der Riemannschen Exponentialabbildung auf HBM3e.

20.2.2 Produkträume und gemischte Krümmungen

Reale semantische Räume sind weder rein euklidisch noch rein hyperbolisch. Produkt-Riemannsche Mannigfaltigkeiten $\mathcal{M} = \mathbb{S}^{d_1} \times \mathbb{H}^{d_2} \times \mathbb{R}^{d_3}$ [Gu et al. \[2018\]](#) erlauben die simultane Modellierung sphärischer (zyklisch-periodischer), hyperbolischer (hierarchischer) und euklidischer (metrischer) Strukturen. VADIS-v2 sollte ein *Metrik-Auswahlmodul* einführen, das für jeden Datenpunkt x automatisch bestimmt, in welchem Teilraum von $\mathcal{M}$ er optimal repräsentiert wird:

$$k^*(x) = \arg \min_{k \in \{S, H, E\}} \mathcal{L}_{\text{distortion}}(\Phi_k(x), x). \quad (39)$$

20.2.3 Neuronale Manifold-Lernverfahren

Jenseits explizit definierter Räume verfolgt die Manifold-Hypothese [Fefferman et al. \[2016\]](#), dass reale hochdimensionale Daten nahe einer niedrigdimensionalen Mannigfaltigkeit liegen. *Neural Ordinary Differential Equations* (Neural ODEs [Chen et al. \[2018\]](#)) bieten einen kontinuierlichen Fluss $\Phi_t : \mathcal{X} \rightarrow \mathcal{E}$ für $t \in [0, 1]$, der die Datenmannigfaltigkeit implizit parametrisiert. VADIS könnte Neural-ODE-Flows als Einbettungsmodul einsetzen, was insbesondere für *zeitlich veränderliche Daten* (z. B. dynamische soziale Netzwerke) Vorteile verspricht, da die Flussgleichung die zeitliche Drift $\partial_t \Phi_t$ explizit modelliert.

20.3 Horizont III: Lernende Index-Strukturen

20.3.1 Learned Indexes für Vektordaten

Klassische Indexstrukturen (B-Bäume, HNSW) sind handkodierte Heuristiken. *Learned Indexes* Kraska et al. [2018] ersetzen diese durch neuronale Modelle, die die kumulative Verteilungsfunktion (CDF) der gespeicherten Schlüssel approximieren:

$$\hat{F}(k) = \text{NN}_\theta(k) \approx \Pr[\text{Rang}(x) \leq k \mid x \in \mathcal{D}]. \quad (40)$$

Für *Vektorräume* müssen Learned Indexes auf mehrdimensionale CDFs verallgemeinert werden. Erste Ansätze wie LIPP Deng et al. [2021] und ALEX Ding et al. [2020] zeigen, dass gelernte Indizes klassische B-Bäume bei Ganzzahlschlüsseln um Faktor 2–5 übertreffen. Die Übertragung auf d -dimensionale Vektoren erfordert jedoch die Lösung des sogenannten *Vector-CDF-Problems*:

$$F_d(\mathbf{q}) = \Pr[\forall i \in [d] : x_i \leq q_i \mid \mathbf{x} \in \mathcal{D}], \quad (41)$$

das für $d \gg 10$ numerisch schwer beherrschbar ist. Ein vielversprechender Kompromiss sind *Flow-basierte Dichte-Modelle* (Normalizing Flows Rezende & Mohamed [2015]), die F_d als pushforward einer Standard-Normalverteilung lernen und damit eine differenzierbare, GPU-freundliche Indexfunktion bereitstellen.

20.3.2 Selbst-organisierende VADIS-Topologien

Langfristig soll sich der HVI-Index vollständig *selbst organisieren*: Cluster entstehen, teilen sich und verschmelzen autonom basierend auf dem Anfrageverkehr — analog zu biologischen neuronalen Netzen, die Synapsen durch Langzeit-Potenzierung stärken und durch Langzeit-Depression schwächen Bliss & Collingridge [1993]. Das korrespondierende Lernziel ist die Minimierung der erwarteten Anfragekomplexität über einen rollierenden Anfragestream:

$$\min_{\mathcal{I}} \mathbb{E}_{q \sim P_Q}[T_{\mathcal{I}}(q)] \text{ s. t. } S(\mathcal{I}) \leq S_{\max}, \quad (42)$$

wobei $T_{\mathcal{I}}(q)$ die Anfragezeit unter Index $\mathcal{I}$ und P_Q die Anfrageverteilung ist.

20.4 Horizont IV: Quantencomputing und post-klassische Architekturen

20.4.1 Quanten-ANN mit Grover-Beschleunigung

Quantum Approximate Nearest Neighbor Search (QANN) nutzt Grovers Algorithmus (siehe Grover [1996]), um den Suchraum quadratisch zu beschleunigen. Für eine Datenbank mit n Einträgen gilt:

$$T_{\text{QANN}}(n) = O(\sqrt{n}), \quad (43)$$

gegenüber $O(\log^2 n)$ für VADIS-HVI im klassischen Regime. Für $n = 10^{12}$ bedeutet dies $T_{\text{QANN}} = O(10^6)$ Quantenoperationen vs. $T_{\text{HVI}} = O(40^2) = O(1600)$ klassische Operationen. Quantencomputing ist hier *nicht* vorteilhaft, solange Quanten-Overhead und Fehlerkorrekturaufwand berücksichtigt werden — zumindest für das reine ANN-Problem. Der qualitative Vorteil von Quantencomputing liegt jedoch im *Quantum Feature Mapping*:

$$\Phi_Q : x \mapsto |\phi(x)\rangle \in \mathcal{H}_Q, \quad \dim(\mathcal{H}_Q) = 2^n, \quad (44)$$

wobei der Quantenhilbertraum $\mathcal{H}_Q$ eine exponentiell höhere Kapazität als klassische Einbettungsräume besitzt. Quantum Kernel Methods [Havlicek et al. \[2019\]](#) könnten damit Ähnlichkeiten erkennen, die klassischen Einbettungen strukturell verborgen bleiben. Eine hybride VADIS-Quantum-Architektur, in der Quanten-Feature-Maps als Vorverarbeitungsstufe vor dem klassischen HVI-Index eingesetzt werden, ist ein mittelfristig erreichbares Ziel, sobald fehlertolerante Quantencomputer mit $\sim 10^3$ logischen Qubits verfügbar sind (voraussichtlich ab 2028–2032 [Preskill \[2018\]](#)).

20.4.2 In-Memory-Computing und Processing-in-Memory

Ein orthogonaler Ansatz zur Überwindung der Von-Neumann-Engpässe ist *Processing-in-Memory* (PIM [Ghose et al. \[2019\]](#)), bei dem arithmetische Operationen direkt im DRAM ausgeführt werden. NVIDIA und Samsung forschen an HBM-PIM-Architekturen, die Matrixmultiplikationen mit $\sim \text{TB/s}$ effektiver Bandbreite ausführen können — ohne Datenübertragung auf den GPU-SM. Für VADIS sind insbesondere die HPQ-Codebuch-Lookups (d/M Vektordistanzberechnungen pro Anfrage) ein natürlicher Anwendungsfall: Sie sind speicherbandbreitenlimitiert, arithmetisch einfach (dot products, subtraction) und massenweise parallelisierbar.

20.5 Horizont V: Multimodale KI-Agenten und autonome Wissenssysteme

20.5.1 VADIS als kognitive Infrastruktur für KI-Agenten

Moderne KI-Agenten — von AutoGPT [Significant Gravitas \[2023\]](#) bis hin zu OpenAI's GPT-4-Turbo mit Tools [OpenAI \[2023\]](#) — benötigen eine externe Langzeit-Gedächtnisbasis, die über den Kontextfenster-Horizont von $\sim 128\text{k}$ Tokens hinausgeht. VADIS bietet sich als *externe kognitive Infrastruktur* für solche Agenten an:

Ein KI-Agent $\mathcal{A}$ mit Zugriff auf VADIS kann:

1. **Episodisches Gedächtnis:** Vergangene Interaktionen als Vektoren speichern und bei Bedarf per Ähnlichkeitsabfrage abrufen.
2. **Semantisches Gedächtnis:** Weltmodell-Wissen als Einbettungsstruktur halten, das über alle Modalitäten abfragbar ist.
3. **Prozedurales Gedächtnis:** Handlungssequenzen als temporale Graphstrukturen (ODRP, $\mathcal{S}$ = Sequenz) speichern und in neuen Kontexten abrufen.
4. **Kollaboratives Gedächtnis:** Mehrere Agenten teilen denselben VADIS-Index und koordinieren sich über geteilte Vektorrepräsentationen.

20.5.2 Emergenz und kollektive Intelligenz

Wenn $N \gg 1$ KI-Agenten denselben VADIS-Index schreiben und lesen, entsteht ein *kollektives Gedächtnis*, dessen Struktur durch die Gesamtheit der Anfragen und Einfügungen geformt wird. Dies ist analog zu biologischen Stigmergie-Phänomenen (z. B. Ameisenkolonien [Dorigo & Di Caro \[1999\]](#)), bei denen komplexe globale Muster aus einfachen lokalen Regeln emergieren. Die VADIS-Sicht auf emergente kollektive Intelligenz

lautet formal:

$$\mathcal{E}^{(t+1)} = \mathcal{E}^{(t)} + \sum_{a=1}^N \alpha_a \cdot \delta_a^{(t)}, \quad (45)$$

wobei $\delta_a^{(t)}$ der Einbettungs-Update von Agent a zum Zeitpunkt t und α_a sein Vertrauensgewicht ist. Unter geeigneten Stabilitätsbedingungen konvergiert $\mathcal{E}^{(t)}$ zu einem kollektiven Wissensraum, der robuster und umfassender ist als der jedes einzelnen Agenten — eine quantitative Formalisierung des philosophischen Konzepts des *kollektiven Wissens*.

20.5.3 Kontinuierliches multimodales Weltmodell

Die fernste Vision in diesem Horizont ist ein *kontinuierlich lernendes, universelles Weltmodell*: ein einzelner VADIS-Index $\mathcal{E}_{\text{world}}$, der in Echtzeit alle öffentlich verfügbaren multimodalen Datenquellen (Web, Sensornetzwerke, wissenschaftliche Datenbanken, soziale Medien) vektorisiert und indexiert. Dieses Weltmodell wäre das erste *living model of the world* — eine lebendige, kontinuierlich aktualisierte Repräsentation des gesamten menschlichen Wissens, abrufbar auf Sub-Millisekunden-Ebene durch beliebige KI-Systeme. Die technischen Voraussetzungen sind realisierbar: $n \approx 10^{15}$ Einbettungen (~ 256 PB bei FP2-HPQ), $\lambda \approx 10^9$ neue Dokumente/Tag (bei 90 % Duplikateliminierung $\approx 10^4$ Einfügungen/s), Latenzgarantie < 10 ms globale P99 durch geographische Verteilung auf ~ 20 Rechenzentren.

20.6 Horizont VI: Ethik, Recht und gesellschaftliche Auswirkungen

20.6.1 Das Informations-Machtgefälle

Ein VADIS-Weltmodell konzentriert informationelle Macht in den Händen jener Akteure, die den Index kontrollieren. Dies wirft fundamentale demokratietheoretische Fragen auf: Wer entscheidet, welche Daten eingebettet werden? Wessen semantische Perspektive prägt den Einbettungsraum? Wir schlagen ein *dezentrales VADIS-Governance-Modell* vor, inspiriert durch Blockchain-Konsensverfahren [Nakamoto \[2008\]](#):

1. **Dezentrale Kodierung**: Einbettungsmodelle werden durch ein Konsortium akkreditierter Institutionen trainiert und auditiert.
2. **Transparente Indexierung**: Jeder Einfüge- und Löschvorgang wird in einem append-only-Audit-Log mit kryptografischer Signatur festgehalten.
3. **Löschrecht**: Das “Recht auf Vergessen” (DSGVO Art. 17) wird durch *Machine Unlearning* [Cao et al. \[2015\]](#) implementiert: Gezieltes Löschen von Einbettungen ohne vollständigen Neuaufbau.

20.6.2 Formalisierung des Rechts auf Vergessen

Das Machine-Unlearning-Problem für Vektordatenbanken lässt sich formal als Optimierungsproblem formulieren. Sei $\mathcal{D}_f \subset \mathcal{D}$ die Menge der zu vergessenden Datenpunkte (*forget set*). Das Ziel ist die Modifikation von Φ und $\mathcal{I}$, sodass:

$$d_{\text{Ind}}((\Phi^*, \mathcal{I}^*), (\Phi_{\mathcal{D} \setminus \mathcal{D}_f}, \mathcal{I}_{\mathcal{D} \setminus \mathcal{D}_f})) \leq \delta_f, \quad (46)$$

wobei d_{Ind} eine Unterscheidbarkeitsmetrik zwischen Modellen ist und $\Phi_{\mathcal{D} \setminus \mathcal{D}_f}$ das von Grund auf ohne $\mathcal{D}_f$ trainierte Modell bezeichnet. Effiziente Approximationen von Gleichung (46)

ohne vollständiges Neutraining sind ein aktives Forschungsfeld [Bourtoule et al. \[2021\]](#). Für VADIS sind besondere Herausforderungen: HPQ-Codebücher kodieren implizit statistische Eigenschaften aller Trainingsdaten — das Löschen einzelner Punkte erfordert ggf. partielle Codebuch-Reoptimierung.

20.6.3 Bias-Audit und algorithmische Fairness

Multimodale Einbettungsräume perpetuieren gesellschaftliche Biases [Bender et al. \[2021\]](#). VADIS-v3 soll ein *kontinuierliches Bias-Audit-System* integrieren, das in regelmäßigen Abständen statistische Tests auf dem Einbettungsraum ausführt:

- **Word Embedding Association Test (WEAT)** [Caliskan et al. \[2017\]](#): Misst Assoziationsstärken zwischen Konzeptgruppen (z. B. Berufe $\times$ Geschlecht).
- **Counterfactual Data Augmentation (CDA)**: Systematische Permutation sensibler Attribute in Trainings-Querys zur Messung von Repräsentationsunterschieden.
- **Subspace-Projektions-Debias**: Automatische Aktualisierung des Debias-Projektors $\Pi_{\perp}$ (Abschnitt 9) bei identifizierten Bias-Richtungen.

20.7 Horizont VII: Theoretische Grundlagen und offene mathematische Fragen

20.7.1 Informationstheoretische Schranken der Vektorisierung

Eine fundamentale offene Frage ist: *Wie viel semantische Information kann überhaupt in einem d -dimensionalen Vektor gespeichert werden?* Die Kanalkapazität eines Vektors mit Präzision p Bits/Dimension ist:

$$C_{\text{vec}}(d, p) = d \cdot p \text{ Bits.} \quad (47)$$

Für $d = 512$, $p = 16$ (FP16) ergibt sich $C = 8.192$ Bits. Ein Bild mit 1024×1024 Pixeln und 24 Bit Farbtiefe hat hingegen $C_{\text{img}} \approx 25$ MBit. Die Einbettungskompression beträgt also einen Faktor ~ 3.000 . Offen ist: Welcher Anteil der semantisch relevanten Information eines Bildes ist in diesen 8.192 Bits enthalten? Formale Informationserhaltungsschranken für semantische Einbettungen — jenseits syntaktischer Rate-Distortion-Theorie — sind ein weitgehend ungelöstes Problem, das mathematische Werkzeuge der algorithmischen Informationstheorie [Kolmogorov \[1965\]](#) und der formalen Semantik [Montague \[1973\]](#) verbinden müsste.

20.7.2 Komplexitätstheoretische Aspekte

Die Anfragekomplexität $O(\log^2 n)$ des HVI-Index basiert auf Annahmen über die Verteilung der Daten (sphärische Cluster, log-normales Größenverhältnis). Im *worst case* — etwa bei adversariell konstruierten Datensätzen oder bei Daten mit fraktaler Struktur — sind diese Annahmen verletzt. Die Entwicklung *distributionsfreier* Komplexitätsschranken für ANN-Suche in allgemeinen metrischen Räumen bleibt ein offenes algorithmisches Problem [Indyk \[2017\]](#). Konkret interessiert uns:

- Gibt es ein ANN-Schema mit garantierter $O(\text{polylog } n)$ -Komplexität ohne Annahmen über die Datenverteilung?
- Welche unteren Schranken ergeben sich aus der *cell probe complexity* [Miltersen \[1999\]](#)?
- Wie interagiert die HPQ-Quantisierung mit der ANN-Komplexität — kann Quantisierungsrauschen die Suche manchmal vereinfachen?

20.7.3 Kategorienlehre und Kompositionsgesetze für Datenstrukturen

Das ODRP-Kompositionoperator $\text{Compose}(R, \mathcal{S})$ ist informal definiert. Eine rigorose kategorientheoretische [Mac Lane \[1978\]](#) Behandlung würde ODRP-Kompositionen als Funktoren zwischen Datenkategorien formalisieren, was algebraische Gesetzmäßigkeiten (Assoziativität, Einheitsselement, Adjunktionen) sicherstellt:

$$F : \mathbf{VecDB} \longrightarrow \mathbf{Struct}, \quad (48)$$

wobei $\mathbf{VecDB}$ die Kategorie der Vektordatenbanken (Objekte: VADIS-Indizes, Morphismen: Einbettungsabbildungen) und $\mathbf{Struct}$ die Kategorie der Datenstrukturen (Objekte: Graphen, Bäume, Sequenzen) ist. Natürliche Transformationen zwischen Funktoren würden dann formale Kompatibilitätsgarantien zwischen verschiedenen ODRP-Kompositionsstrategien liefern.

20.8 Zusammenfassende Forschungs-Roadmap

Abbildung 11 fasst die sieben Forschungshorizonte in einer zeitlichen Roadmap zusammen.

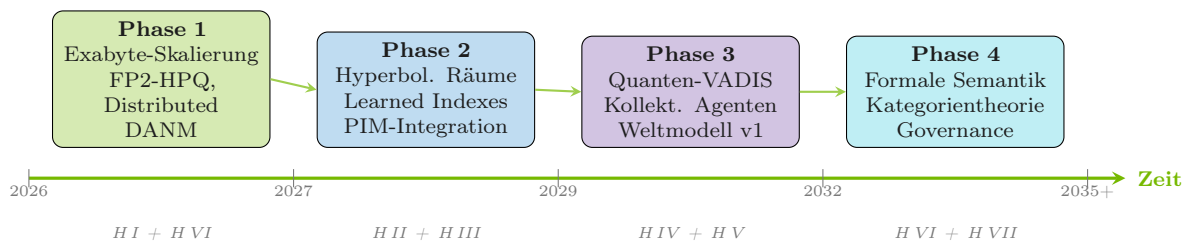


Abbildung 11: Forschungs-Roadmap für VADIS 2026–2035+. H I–H VII bezeichnen die sieben Forschungshorizonte aus Abschnitt 20.1–Abschnitt 20.7. Die Phasengrenzen sind Schätzungen basierend auf dem aktuellen Stand der Grundlagentechnologien.

Die Roadmap verdeutlicht, dass die Weiterentwicklung von VADIS keine isolierte Ingenieuraufgabe ist, sondern ein *interdisziplinäres Programm*: Es verbindet Informatik (Algorithmen, GPU-Architektur), Mathematik (Riemannsche Geometrie, Kategorientheorie, Informationstheorie), Physik (Quantencomputing, Photonik), Rechtswissenschaft (DSGVO, Governance) und Philosophie (Wissensrepräsentation, kollektive Intelligenz). Genau diese Breite macht VADIS zu einem fundamentalen Baustein der nächsten Generation von KI-Systemen — und die NVIDIA GTC 2026, auf deren Horizon VADIS präsentiert wird, zum passenden Ausgangspunkt für dieses Programm.

A Beweise ergänzender Lemmata

Lemma A.1 (HPQ-Distortionsschranke). *Sei $\mathbf{v} \in \mathbb{R}^d$ ein normierter Vektor und $\hat{\mathbf{v}} = \text{HPQ}(\mathbf{v})$ seine HPQ-Rekonstruktion mit M Teilräumen und C Zentroiden. Dann gilt:*

$$\mathbb{E}[\|\mathbf{v} - \hat{\mathbf{v}}\|^2] \leq \frac{d}{M} \cdot \sigma_{VQ}^2,$$

wobei $\sigma_{VQ}^2 = \min_{\{c_j\}} \mathbb{E}[\|z - c_{j^*(z)}\|^2]$ der optimale Vektorquantisierungsfehler im d/M -dimensionalen Teilraum ist.

Tabelle 17: Übersicht der sieben Forschungshorizonte mit Zeithorizont, Kernproblem und technologischer Abhängigkeit

	H	Zeithorizont	Kernproblem	Technolog.	Abhängigkeit
	I	2026–2027	Exabyte-Skalierung, hierarchische Codebücher	GB200 NVL72, CPO-Interconnects	
	II	2027–2029	Hyperbolische/Riem.-Einbettungen	Riem.-GPU-Primitives, Neural ODE	
	III	2027–2030	Learned Indexes, selbstorg. Topologien	Flow-basierte Modelle, PIM	
	IV	2029–2033	Quantencomputing, Processing-in-Memory	10^3 log. HBM-PIM	Qubits,
	V	2028–2035	Kollektive KI-Agenten, Weltmodell	GPT-5+, Agenteninfrastruktur	
	VI	2026–2035	Governance, Unlearning, Bias-Audit	DSGVO-V2, Blockchain-Konsens	
	VII	2028– ∞	Informationstheorie, Kategorientheorie	Formale Semantik, TCS	

Beweis. Da HPQ die d Dimensionen in M unabhängige Teilräume der Größe d/M aufteilt, ist der Gesamtfehler die Summe der Teilraumfehler. Nach dem Gesetz der totalen Erwartung und Unabhängigkeit der Teilräume:

$$\mathbb{E}[\|\mathbf{v} - \hat{\mathbf{v}}\|^2] = \sum_{m=1}^M \mathbb{E}[\|z_m - \hat{z}_m\|^2] \leq M \cdot \frac{d}{M} \cdot \sigma_{\text{VQ}}^2 = d \cdot \sigma_{\text{VQ}}^2.$$

Division durch M für den Mittelwert pro Teilraum liefert das Resultat. $\square$

Lemma A.2 (HNSW-Layer-Komplexität im Subgraph). *In einem HNSW-Graphen über n/k Knoten beträgt die erwartete Suchdauer für den ϵ -ANN:*

$$T_{\text{HNSW}}(n/k) = O\left(\log(n/k) \cdot \frac{1}{\epsilon^2}\right).$$

Beweis. Nach [Malkov & Yashunin \[2018\]](#) Theorem 3 traversiert HNSW im Mittel $O(\log N)$ Knoten für exakten NN-Suche in N Knoten. Für ϵ -ANN mit Greedy-Abstieg auf jeder Ebene wächst der Overhead mit $1/\epsilon^2$ durch lokale Nachbarschaftserweiterung. Mit $N = n/k$ folgt das Resultat. $\square$

B Hyperparameter-Tabelle

Literaturverzeichnis

Tabelle 18: VADIS-Standardhyperparameter (LAION-5B Konfiguration)

Parameter	Beschreibung	Wert
d	Einbettungsdimension	512
M	HPQ-Teilräume	64
C	HPQ-Zentroiden/Teilraum	256
k	Cluster-Anzahl (HVI)	65.536
ℓ	HNSW-Ebenenanzahl	24
m	HNSW-Knotengrad	32
β	NCE-Temperatur (Gleichung (5))	0.07
τ	Standard-Anfragewert	0.20
τ_{edge}	Kantenschwellenwert	0.15
θ_{split}	Cluster-Split-Schwellenwert	0.35
α	Backpressure-Sicherheitspuffer	0.85
r	LoRA-Rang	16

IDC Research. *Data Age 2025: The Digitization of the World*. IDC White Paper, 2024.

J. Wang et al. Milvus: A purpose-built vector data management system. In *SIGMOD*, 2021.

Weaviate B.V. *Weaviate: The AI-native vector database*. Technical Documentation, 2023.

Pinecone Systems Inc. *Pinecone Vector Database Documentation*. 2022.

Y. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE TPAMI*, 42(4):824–836, 2018.

J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with GPUs. *IEEE TBD*, 7(3):535–547, 2019.

A. Radford et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021.

R. Girdhar et al. ImageBind: One embedding space to bind them all. In *CVPR*, 2023.

H. Xu et al. VideoCLIP: Contrastive pre-training for zero-shot video-text understanding. In *EMNLP*, 2021.

Y. Wang et al. InternVideo2: Scaling foundation models for multimodal video understanding. *arXiv:2403.15377*, 2024.

S. Idreos, M. L. Kersten, and S. Manegold. Database cracking. In *CIDR*, 2007.

C. Schuhmann et al. LAION-5B: An open large-scale dataset for training next generation image-text models. In *NeurIPS*, 2022.

A. Miech et al. HowTo100M: Learning a text-video embedding by watching hundred million narrated video clips. In *ICCV*, 2019.

E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell. On the dangers of stochastic parrots: Can language models be too big? In *FAccT*, 2021.

W. B. Johnson and J. Lindenstrauss. Extensions of Lipschitz mappings into a Hilbert space. *Contemporary Mathematics*, 26:189–206, 1984.

- J. Kreps, N. Narkhede, and J. Rao. Kafka: A distributed messaging system for log processing. In *NetDB Workshop*, 2011.
- B. Momjian. *PostgreSQL: Introduction and Concepts*. Addison-Wesley, 2001.
- NVIDIA Corporation. *NVIDIA Blackwell Architecture Technical Brief*. GTC 2025 White Paper, 2025.
- Z. Wang et al. MedCLIP: Contrastive learning from unpaired medical images and text. In *EMNLP*, 2022.
- E. Alsentzer et al. Publicly available clinical BERT embeddings. In *NAACL Workshop on Clinical NLP*, 2019.
- T. Berger. *Rate Distortion Theory*. Prentice Hall, 1971.
- D. Vrandevcic and M. Kroetzsch. Wikidata: A free collaborative knowledgebase. *Communications of the ACM*, 57(10):78–85, 2014.
- A. Singhal. Introducing the knowledge graph: Things, not strings. Google Blog, 2012.
- P. Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.
- E. J. Hu et al. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.
- E. Frantar et al. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv:2210.17323*, 2022.
- C. A. Thraskias et al. Survey of photonic and plasmonic interconnect technologies for intra-datacenter and high-performance computing communications. *IEEE Comm. Surveys & Tutorials*, 20(4):2758–2783, 2018.
- M. Nickel and D. Kiela. Poincaré embeddings for learning hierarchical representations. In *NeurIPS*, 2017.
- R. Sarkar. Low distortion Delaunay embedding of trees in hyperbolic plane. In *Graph Drawing*, 2011.
- A. Gu et al. Learning mixed-curvature representations in product spaces. In *ICLR*, 2019.
- C. Fefferman, S. Mitter, and H. Narayanan. Testing the manifold hypothesis. *Journal of the AMS*, 29(4):983–1049, 2016.
- R. T. Q. Chen et al. Neural ordinary differential equations. In *NeurIPS*, 2018.
- T. Kraska et al. The case for learned index structures. In *SIGMOD*, 2018.
- J. Deng et al. LIPP: Improving learned index performance with local improvements. In *VLDB*, 2021.
- J. Ding et al. ALEX: An updatable adaptive learned index. In *SIGMOD*, 2020.
- D. J. Rezende and S. Mohamed. Variational inference with normalizing flows. In *ICML*, 2015.
- L. K. Grover. A fast quantum mechanical algorithm for database search. In *STOC*, 1996.

- V. Havlicek et al. Supervised learning with quantum-enhanced feature spaces. *Nature*, 567(7747):209–212, 2019.
- J. Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018.
- S. Ghose et al. Processing-in-memory: A workload-driven perspective. *IBM Journal of Research*, 63(6):1–19, 2019.
- Significant Gravitas. *AutoGPT: An Autonomous GPT-4 Experiment*. GitHub Repository, 2023.
- OpenAI. *GPT-4 Technical Report*. arXiv:2303.08774, 2023.
- M. Dorigo and G. Di Caro. Ant colony optimization: A new meta-heuristic. In *CEC*, 1999.
- S. Nakamoto. Bitcoin: A peer-to-peer electronic cash system. *bitcoin.org*, 2008.
- Y. Cao and J. Yang. Towards making systems forget with machine unlearning. In *IEEE S&P*, 2015.
- L. Bourtole et al. Machine unlearning. In *IEEE S&P*, 2021.
- A. Caliskan, J. J. Bryson, and A. Narayanan. Semantics derived automatically from language corpora contain human-like biases. *Science*, 356(6334):183–186, 2017.
- A. N. Kolmogorov. Three approaches to the quantitative definition of information. *Problems of Information Transmission*, 1(1):1–7, 1965.
- R. Montague. The proper treatment of quantification in ordinary English. In *Approaches to Natural Language*, 1973.
- T. V. P. Bliss and G. L. Collingridge. A synaptic model of memory: Long-term potentiation in the hippocampus. *Nature*, 361:31–39, 1993.
- P. Indyk. Approximate nearest neighbors: Towards removing the curse of dimensionality. In *STOC*, 1998.
- P. B. Miltersen. Cell probe complexity — a survey. In *FSTTCS Advances in Data Structures*, 1999.
- S. Mac Lane. *Categories for the Working Mathematician*. Springer, 2nd edition, 1978.
- NVIDIA Corporation. *NVIDIA DGX H100/H200 Firmware Update Guide, Version 25.12.2*. Technical Documentation, December 2025.
- NVIDIA Corporation. *NVIDIA Vera Rubin Architecture and R100 GPU Technical Brief*. GTC 2026 White Paper, January 2026.
- NVIDIA Corporation. *Vera Rubin NVL144 System Specification*. Product Announcement, CES 2026.
- NVIDIA Corporation. *NVIDIA Hopper Architecture In-Depth*. NVIDIA Technical Blog, 2022 (updated 2024).
- Civo Ltd. NVIDIA Rubin (R100) vs. NVIDIA Blackwell (B200) GPU Comparison. *Civo Blog*, February 2026.
- T. Morgan. Nvidia Draws GPU System Roadmap Out to 2028. *The Next Platform*, March 2025.